

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

9. Ember-gép kapcsolat eszközök

Írta: Kulcsár Dávid

Lektorálta: Hadarics Szabolcs

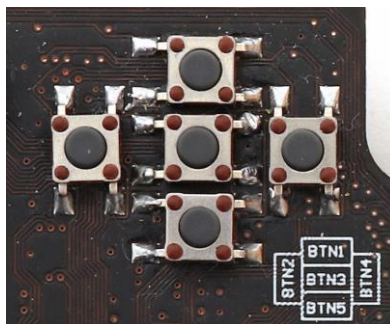
Teremtsünk kapcsolatot az elektronikus eszközeinkkel! A gépek akkor könnyítik meg a mindennapjainkat, ha azt csinálják, amit mi szeretnénk. Ehhez viszont arra van szükség, hogy felügyeljük a működésüket, beavatkozzunk a folyamataikba és irányítsuk őket. Ezt az információátadást segítik elő az ember-gép interfészek (az angol szakirodalomban a Human-Machine Interface, röviden HMI kifejezéssel találkozhatunk).

Mi kell ahhoz, hogy a készülék és felhasználója között interakció jöjjön létre? Mivel ez egy kétirányú kommunikációs kapcsolat, ezért az eszközt olyan hardveres perifériákkal kell felruházni, melyek ezt támogatják. A gép szemszögéből nézve szükségünk van úgynevezett bemenetekre és kimenetekre. A bemenetek teszik lehetővé az adatok bevitelét, ezek adják kezünkbe az irányítás kulcsát. Információhoz pedig a kimeneteken keresztül jutunk, belelátva ezzel az elektronika "lelkivilágába". Természetesen a hardver önmagában nem elég a megfelelő működés eléréséhez, elengedhetetlen a rendszert életre keltő szoftver.

BEMENETI ESZKÖZÖK

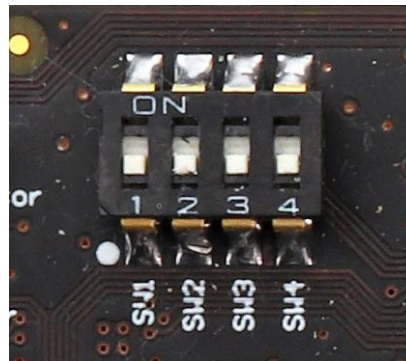
A leggyakrabban használt bemeneti interfészek megtalálhatóak a fejlesztői csomagban, tekintsük át ezeket.

5 darab nyomógomb található a fejlesztőkártyán kereszt alakban elhelyezve, hogy szemléletesen megvalósítható legyen általuk egy menürendszer kezelése. A szélső gombok lehetnek a „nyíl” billentyűk, vagyis a navigációra szolgáló fel-, le-, balra-, jobbra gombok, míg a középső a végrehajtást engedélyező OK gomb.



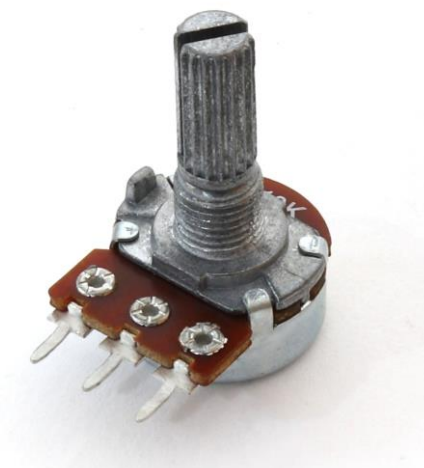
1. ábra: Fejlesztőpanelen található nyomógombok

A kártyán található még 4 darab tolókapcsoló.



2. ábra: Fejlesztőpanelen lévő DIP kapcsoló

Valamint 1 darab potenciométer jár még a csomaghoz, melyet szintén használhatunk.



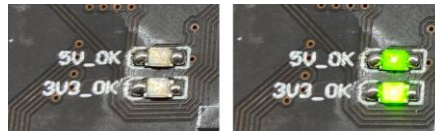
3. ábra: Forgó potenciométer

Ezekkel a perifériákkal már vagy találkozottok, vagy egy későbbi tananyagrészen mutatjuk be használatukat, ezért részletesen nem térek ki rájuk. Ha mélyebben érdekelnek a kapcsolóelemek (nyomógombok, kétállású kapcsolók, forgókapcsolók stb.), akkor ajánlom figyelmedbe a Kristálytisza Elektronika első részének 07 Kapcsolóeszközök című tananyag részét, melyben sok hasznos információt találsz a témával kapcsolatban.

KIMENETI ESZKÖZÖK

A legalapvetőbb kimeneti elem, amely segítségével információhoz juthatunk, az a LED (angol nevén Light-Emitting Diode). Többféle információtartalommal is bírhat használatától függően.

Legegyszerűbb jelzés lehet az, hogy világít-e. Két ilyen célt szolgáló LED is található a fejlesztőkártyán, amelyek azt mutatják számunkra, hogy a belső 5 V-os és 3.3 V-os tápfeszültségek elő lettek állítva.

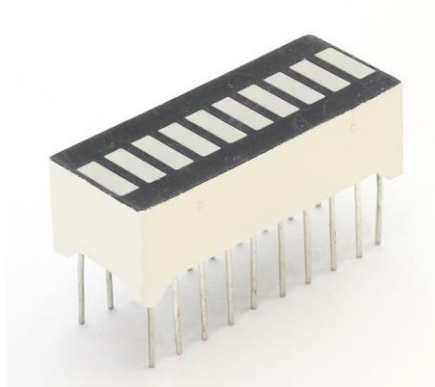


4. ábra: Tápellátás megfelelőségét visszajelző LED-ek

Visszajelzés adható különböző frekvenciájú villogással, de akár a fényerőt is használhatjuk. Egy másik kedvelt megoldás a többszínű fényforrás, melyet RGB LED-nek (Red-Green-Blue LED) nevezünk. Státuszinformációra például kiváló, hiszen az egyes állapotok jól reprezentálhatóak a különböző színekkel. Egy RGB LED a piros-zöld-kék színeket és az ezekből kikeverhető további színekombinációkat tudja produkálni.

Ha több információt szeretnénk közölni a felhasználó felé, akkor több LED-et is használhatunk, és mindegyikhez egyértelműen hozzárendeljük, hogy mit jelez. Gondoljunk csak egy routerre, hogy hány LED van rajta és mindegyik valamilyen jelentéssel bír!

Annak érdekében, hogy komplexebb adatokat is meg tudjunk jeleníteni, helyezzük el a LED-eket valamilyen struktúra szerint, így elérve egy sokkal vizuálisabb megjelenítést. Kitalálhatunk olyan megoldásokat például a PWM kitöltési tényezőjének százalékos beállítását több, egymás mellé sorban elhelyezett LED segítségével vizualizáljuk. (Kapható egyébként ilyen periféria a piacon LED sáv néven, angolul LED bar.)



5. ábra: LED sáv áramköri elem

A LED-ek használatára már láttatok példákat, gondolatok csak a FutóFény projektre vagy a nyomógombok és LED-ek használatát bemutató projektre.

KIJELZŐK

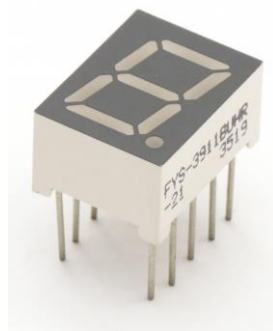
Ha továbbvisszük azt a gondolatmenetet, hogy struktúrába rendezett LED-ek segítségével bonyolultabb információk átadását is megvalósíthatjuk, akkor rövid idő alatt eljutunk a LED kijelzők világába. Ezek az eszközök már számok, betűk megjelenítését teszik lehetővé számunkra. Még mielőtt részletesebben kitérnénk a LED-es megjelenítőkre, nézzük meg az egyszerűbb kijelzők típusait és azok csoportosítását:

- LED kijelzők
 - 7-szegmens kijelző
 - Alfa-numerikus kijelző
 - Pontmátrix kijelző
- LCD kijelzők
 - 7-szegmens LCD kijelző
 - Alfa-numerikus LCD kijelző
 - Grafikus LCD kijelző
- OLED kijelzők
 - Alfa-numerikus OLED kijelző
 - Grafikus OLED kijelző

LED KIJELZŐK

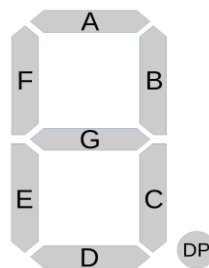
7-szegmenses kijelző

Egyszerűsége miatt a kedvelt alkatrészek közé tartozik a hétszegmenses kijelző. Ez a megjelenítőeszköz funkcióját tekintve az arab számok megjelenítésére alkalmas.



6. ábra: 7-szegmenses kijelző

Ha megvizsgáljuk a felépítését, minden egyes szegmenshez egy-egy LED tartozik. A szegmens jelölése állandó a hétszegmenses kijelzők világában, mely a következő:



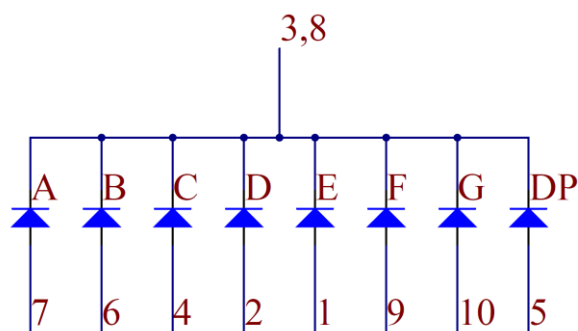
7. ábra: 7-segmeneses kijelző felépítése

A megfelelő szegmensek be- és kikapcsolásával tudjuk az arab számokat megjeleníteni, melynek összefoglaló táblázatát itt találjátok (pontszegmens nélküli verzió):

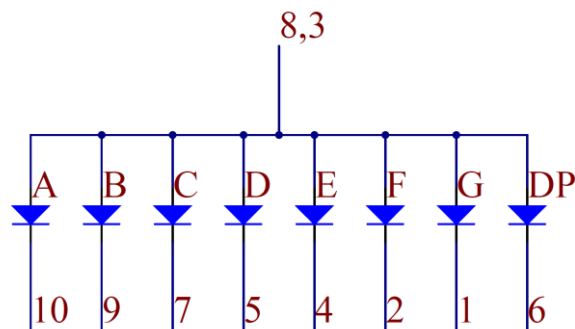
	A	B	C	D	E	F	G
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1

8. ábra: Számok megjelenítése a 7-segmeneses kijelzőn,
1-es jelzi a bekapcsolt szegmenst, 0-s a kikapcsolt szegmenst

A szegmensekhez tartozó LED-ek kapcsolása típusonként eltérő, ezért mindig meg kell nézni, hogy milyen elrendezésbe vannak kötve. Léteznek olyan megjelenítők, melyeknél az anód lábak vannak közösítve, és vannak olyan típusok, melyeknél a katód lábak vannak azonos potenciálon.



9. ábra: Közös katódos LED felépítés



10. ábra: Közös anódos LED felépítés

Ez szoftveres szempontból is fontos számunkra, hiszen a LED-ek vezérlése ennek figyelembevételével történik. A LED-eket kapcsolhatjuk egyesével, az eddig megszokott módon. Ennek a megoldásnak a hátránya, hogy sok GPIO láb kell a működtetéséhez, előnye viszont, hogy nem kell plusz hardveres alkatrész (a szoftver ingyen van viccet biztosan ismeritek). Ha nincs elég GPIO lába a mikrokontrollerünknek, akkor be kell iktatni egy meghajtóáramkört, melyet BCD-to-7-segment driver/decoder-nek (Binary Coded Decimal) hívnak. Ez a binárisan kódolt decimális formátumból állít elő a hétszegmenses kijelző vezérlésére alkalmas jeleket. A binárisan kódolt decimális szám előállításához a tízes számrendszer helyiértékeit használjuk, melynek során 4 bitet rendelünk egy helyiértékhez, mivel a legnagyobb előforduló szám a tízes számrendszerben a 9 ezért szükséges a 4 bit. Azért használjuk mivel sokkal gyorsabban és hatékonyabban tudunk így a 10-es számrendszerből átváltani, mintha ugyanezt binárisba tennénk. Például a 19-et a "0001 1001" fogja reprezentálni. A legegyszerűbb elmagyarázni ezt egy példán keresztül, melyhez az SN74LS47 nevű integrált áramkört választottam:

'46A, '47A, 'LS47 FUNCTION TABLE (T1)

DECIMAL OR FUNCTION	INPUTS						$\overline{BI}/\overline{RBO}^\dagger$	OUTPUTS							NOTE
	$\overline{LT}$	$\overline{RBI}$	D	C	B	A		a	b	c	d	e	f	g	
0	H	H	L	L	L	L	H	ON	ON	ON	ON	ON	ON	OFF	1
1	H	X	L	L	L	H	H	OFF	ON	ON	OFF	OFF	OFF	OFF	
2	H	X	L	L	H	L	H	ON	ON	OFF	ON	ON	OFF	ON	
3	H	X	L	L	H	H	H	ON	ON	ON	ON	OFF	OFF	ON	
4	H	X	L	H	L	L	H	OFF	ON	ON	OFF	OFF	ON	ON	
5	H	X	L	H	L	H	H	ON	OFF	ON	ON	OFF	ON	ON	
6	H	X	L	H	H	L	H	OFF	OFF	ON	ON	ON	ON	ON	
7	H	X	L	H	H	H	H	ON	ON	ON	OFF	OFF	OFF	OFF	
8	H	X	H	L	L	L	H	ON	ON	ON	ON	ON	ON	ON	
9	H	X	H	L	L	H	H	ON	ON	ON	OFF	OFF	ON	ON	
10	H	X	H	L	H	L	H	OFF	OFF	OFF	ON	ON	OFF	ON	
11	H	X	H	L	H	H	H	OFF	OFF	ON	ON	OFF	OFF	ON	
12	H	X	H	H	L	L	H	OFF	ON	OFF	OFF	OFF	ON	ON	
13	H	X	H	H	L	H	H	ON	OFF	OFF	ON	OFF	ON	ON	
14	H	X	H	H	H	L	H	OFF	OFF	OFF	ON	ON	ON	ON	
15	H	X	H	H	H	H	H	OFF	OFF	OFF	OFF	OFF	OFF	OFF	
$\overline{BI}$	X	X	X	X	X	X	L	OFF	OFF	OFF	OFF	OFF	OFF	OFF	2
$\overline{RBI}$	H	L	L	L	L	L	L	OFF	OFF	OFF	OFF	OFF	OFF	OFF	3
$\overline{LT}$	L	X	X	X	X	X	H	ON	ON	ON	ON	ON	ON	ON	4

11. ábra: Az SN74LS47 funkcionális táblája

Forrás: Texas Instruments

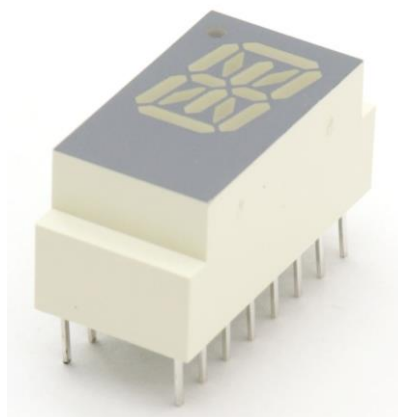
https://www.ti.com/lit/ds/symlink/sn74ls47.pdf?ts=1614106680045&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FSN74LS47

Az adatlapban található táblázat jól szemlélteti a működést. A kijelzésre szánt decimális értéket fel kell írunk bináris formában (4 biten), és a bitek logikai értékének megfelelő feszültségszintet kell az IC bemeneteire kapcsolni. A kimeneten már a dekódolt hétszegmenses kijelző vezérlőjeleit fogjuk látni. Példának válasszuk a decimális 5-öt, mely binárisan felírva 0b0101. A bemenetekre tehát alacsony (D) - magas (C) - alacsony (B) - magas (A) feszültségszinteket kell kapcsolni. Ebben az esetben a kimeneten a következő szegmensek lesznek bekapcsolva: a-c-d-f-g.

Ennél a kijelzőtípusnál elég nagy kínálattal találkozhatunk, léteznek mindenféle színben, méretben, változó lábkiosztással, változó tulajdonságú LED-ekkel és még sorolhatnám.

Alfanumerikus kijelző

A felturbózott 7-szegmenses kijelző.



12. ábra: Alfa numerikus kijelző

Ahogy azt az ábrán is láthatjuk, az alfanumerikus kijelző alapja a 7-szegmenses kijelző. Különbség, hogy a vízszintes szegmensek két részre lettek osztva, valamint további 3-3 szegmessel egészült ki a felső és az alsó tartomány. Ezekkel a módosításokkal és kiegészítésekkel már képesek vagyunk az angol ábécé betűinek a megjelenítésére is. Vezérlése alapvetően hasonlít a korábban bemutatotthoz (LED-ek vezérlése), azonban részletes bemutatást most nem végzünk.

Pontmátrix kijelző

Ennél a kijelző típusnál a LED-ek pontrácsként vannak elhelyezve, összekötve pedig oszloponként és soronként vannak. Vezérlésük ebből fakadóan oszlop- és sorválasztós logikával kivitelezhető. Itt is rengetegféle típussal találkozhatunk (méret, LED szám, LED sűrűség stb.). Előnye, hogy a megjelenítendő információ nem korlátozódik a számok és betűk világára, ezt használva már bonyolultabb alakzatokat, képeket tudunk rajzolni.



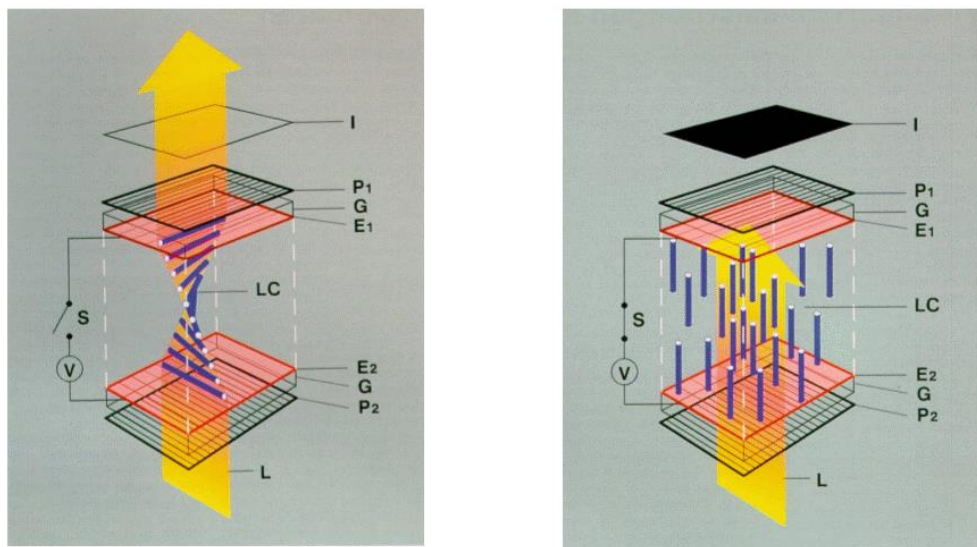
13. ábra: Pontmátrix kijelző

LCD KIJELZŐ

A folyadékkristályos kijelző ismertebb elnevezése az LCD kijelző, mely az angol Liquid Crystal Display név rövidítéséből ered. Felépítésüket és működésüket a következő ábra segítségével szeretném elmagyarázni.

Egy képpont a két átlátszó lap között elhelyezkedő folyékony kristályos anyagból, két elektródából, egy első- és egy hátsó polarizációs rétegből, valamint egy fényforrásból áll.

Röviden és egyszerűen bemutatva a működést: alapesetben (ha az elektródákra nem kapcsolunk feszültséget) a folyadékkristály csavart állapotot vesz fel, ami segíti a fényforrásból érkező fény eljutását a képpont látható részére (ezt az esetet láthatjuk a bal oldali ábrán). Ekkor az adott pixelt világosnak fogjuk látni. Abban az esetben viszont, ha az elektródákra feszültséget kapcsolunk, akkor a folyadékkristály szerkezete megváltozik, melynek eredményeképpen a fény nem tud eljutni a kimenetre (ezt az esetet láthatjuk a jobb oldali ábrán).



14. ábra: LCD kijelző rétegei

Forrás: Panja-san [CC BY-SA 3.0 (<https://creativecommons.org/licenses/by-sa/3.0/>)]
<https://commons.wikimedia.org/wiki/File:TN-LCD-schematic-MS-208kB.png>

Az ábrán látható kifejezések jelentése a következő:

- I: megjelenő kép,
- P: polarizátor rétegek,
- G: üveg rétegek,
- E: elektródák,
- LC: folyadékkristály,
- L: fény,
- S: kapcsoló
- V: feszültségforrás.

A bal oldali kép szemlélteti a bekapcsolt, míg a jobb oldali a kikapcsolt állapotot.

A működés teljes és részletes bemutatása túlmutat ezen a fejezeten, mivel ehhez további ismeretekre lenne szükség, mint például, a fény felírása hullámként vagy másik példa a polarizáció működése és hatása.

Széleskörben használt megjelenítőeszközkőről beszélünk, ugyanis olcsó, könnyű előállítani, és jól használható direkt napfényben is a háttérvilágításnak köszönhetően. Hátránya is ebből ered, ugyanis a hátsó megvilágítás miatt kevésbé pontosan adja vissza a színeket.

7-szegmenses LCD kijelző

Ennél a típusnál csupán annyit jegyeznek meg, hogy a megjeleníthető karakterek megegyeznek a hagyományos hétszegmenses kijelző karaktereivel. A technológiai különbségeken felül a vezérlésük is változott, melyet a következő típusnál mutatok be.

Alfanumerikus LCD kijelző

A legkedveltebb LCD típus, ugyanis kevés energiabefektetéssel gyors eredményt érhetünk el vele. Alfanumerikus, tehát csak rögzített karakterképet tudunk a segítségével megjeleníteni, de ezek lehetnek betűk, számok és írásjelek is. Univerzálisabb ezáltal a 7-szegmenses típusnál, valamint az esetek többségében elegendő a kívánt funkcionalitás eléréséhez.



15. ábra: Alfanumerikus LCD kijelző

Ezek a típusok általában a HD44780 vezérlővel vagy valamilyen ezzel kompatibilis controllerrel vannak ellátva, ugyanis a Hitachi HD44780-as LCD vezérlője ipari szabvánnyá nőtte ki magát az idők során. A

szabvány létrejöttét követően minden gyártó törekedett a kompatibilitásra az LCD modulok lábkiosztását, méretét és utasításkészletét tekintve. Ennek köszönhető a hasonló és egyszerű használatuk, valamint a csereszabotosság.

Az LCD modulok lábkiosztását tekintve 3 táp- és 2 háttérvilágítás tápellátására szolgáló láb áll rendelkezésünkre, vezérlésre pedig 8 adatvonalat és 3 vezérlőlábat használhatunk. A vezérlés módjáról az adatlapban közölt utasításkészletből tudunk tájékozódni:

COMMAND	COMMAND CODE										COMMAND CODE	E-CYCLE f _{osc} =250KHz
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0		
SCREEN CLEAR	0	0	0	0	0	0	0	0	0	1	Screen Clear, Set AC to 0 Cursor Reposition	1.64ms
CURSOR RETURN	0	0	0	0	0	0	0	0	1	*	DDRAM AD=0, Return, Content Changeless	1.64ms
INPUT SET	0	0	0	0	0	0	0	1	I/D	S	Set moving direction of cursor, Appoint if move	40us
DISPLAY SWITCH	0	0	0	0	0	0	1	D	C	B	Set display on/off,cursor on/off, blink on/off	40us
SHIFT	0	0	0	0	0	1	S/C	R/L	*	*	Remove cursor and whole display,DDRAM changeless	40us
FUNCTION SET	0	0	0	0	1	DL	N	F	*	*	Set DL,display line,font	40us
CGRAM AD SET	0	0	0	1	ACG					Set CGRAM AD, send receive data		40us
DDRAM AD SET	0	0	1	ADD					Set DDRAM AD, send receive data		40us	
BUSY/AD READ CT	0	1	BF	AC					Executing internal function, reading AD of CT		40us	
CGRAM/ DDRAM DATA WRITE	1	0	DATA WRITE					Write data from CGRAM or DDRAM		40us		
CGRAM/ DDRAM DATA READ	1	1	DATA READ					Read data from CGRAM or DDRAM		40us		
	I/D=1: Increment Mode; I/D=0: Decrement Mode S=1: Shift S/C=1: Display Shift; S/C=0: Cursor Shift R/L=1: Right Shift; R/L=0: Left Shift DL=1: 8D DL=0: 4D N=1: 2R N=0: 1R F=1: 5x10 Style; F=0: 5x7 Style BF=1: Execute Internal Function; BF=0: Command Received										DDRAM: Display data RAM CGRAM: Character Generator RAM ACG: CGRAM AD ADD: DDRAM AD & Cursor AD AC: Address counter for DDRAM & CGRAM	E-cycle changing with main frequency. Example: If fcp or f _{osc} =270KHz 40us x 250/270 =37us

16. ábra: Alfa numerikus kijelző vezérlési módszere

Forrás: WaveShare

https://www.waveshare.com/datasheet/LCD_en_PDF/LCD1602.pdf

Szerencsére az elterjedt használata miatt sok előre megírt szoftveres könyvtár található hozzá, melyek használata gyorsabbá teszi a projektek fejlesztését.

Grafikus LCD kijelző

A grafikus LCD kijelzők annyiban különböznek az előző típusoktól, hogy a megjelenítő képpontjai szabadon, kötetlenül vezérelhetők, akár már képeket is meg tudunk velük jeleníteni. Ez sokkal nagyobb szabadságot jelent a felhasználó számára, viszont a kívánt funkció elérése jóval több időt vesz igénybe.

OLED KIJELZŐ

Az OLED az angol Organic Light-Emitting Diode kifejezés rövidítése, mely szerves fénykibocsátó diódát jelent. Ez a három szó tömören össze is foglalja ennek a technológiának a működési elvét: léteznek olyan szerves anyagok, melyek feszültség hatására fényt bocsátanak ki magukból, és ezt használják ki az OLED-panelek esetén. Sok előny köszönhető annak, hogy nem szükséges háttérvilágítás:

- a kijelzőknél ismert rétegzett struktúra egyszerűbb szerkezetűvé válik, így könnyebb és vékonyabb megjelenítőket kapunk eredményül
- energiatakarékos
- tökéletes fekete szín érhető el, így a kontraszt is kiváló minőségű lesz
- pontos, stabil színei vannak, nincs torzítás
- kiváló a betekintési szöge (látószög)

Érdekesség, hogy az OLED kijelzők működési elve a szentjánosbogaraktól származik.

Az OLED kijelzőknek is létezik alfanumerikus változata, mivel azonban a működési elvükre korábban már kitértünk, ezért itt nem részletezzük újra. Inkább ismerkedjünk meg a grafikus kijelzőkkel, és nézzük meg működésüket a gyakorlatban közösen.

GRAFIKUS OLED KIJELZŐ A GYAKORLATBAN

A továbbiakban a grafikus OLED kijelzők használatát fogjuk megvizsgálni a gyakorlatban a fejlesztőcsomagunk segítségével. Ebben találtok egy általunk választott megjelenítőt, melyet közösen fogunk életre kelteni. Lássunk is hozzá!

Hogyan csatlakoztassuk a kijelzőt a fejlesztőpanelhez? Ehhez a feladathoz némi információra van szükségünk a mellékelt adatlapból, melyben található egy táblázat a lábak számozásáról, elnevezéséről és funkciójáról:

Interface Pin Function

No.	Symbol	Function
1	VCC	Power supply input: 2.8 – 5.2V
2	GND	Ground
3	SCL	This is the serial clock input
4	SDA	This is a serial data input

17. ábra: Általunk használt OLED kijelző lábkiosztása

Forrás: Midas Displays

<https://www.midasdisplays.com/getattachment/785afe2f-1212-4e1f-8ab9-530d4a931614/specification>

Ebből kiderül, hogy egy I²C kommunikációval rendelkező perifériával van dolgunk, mely képes 3,3 V-ról működni. A következő kérdés az eszköz áramfelvétele, vagyis az, hogy van-e szükségünk külső tápra a használatához, vagy elég az 5 V-os USB bemenet a 3,3 V-os tápáramkör meghajtására. Ezek az információk is megtalálhatóak egy másik táblázat soraiban:

DC Electrical Characteristics

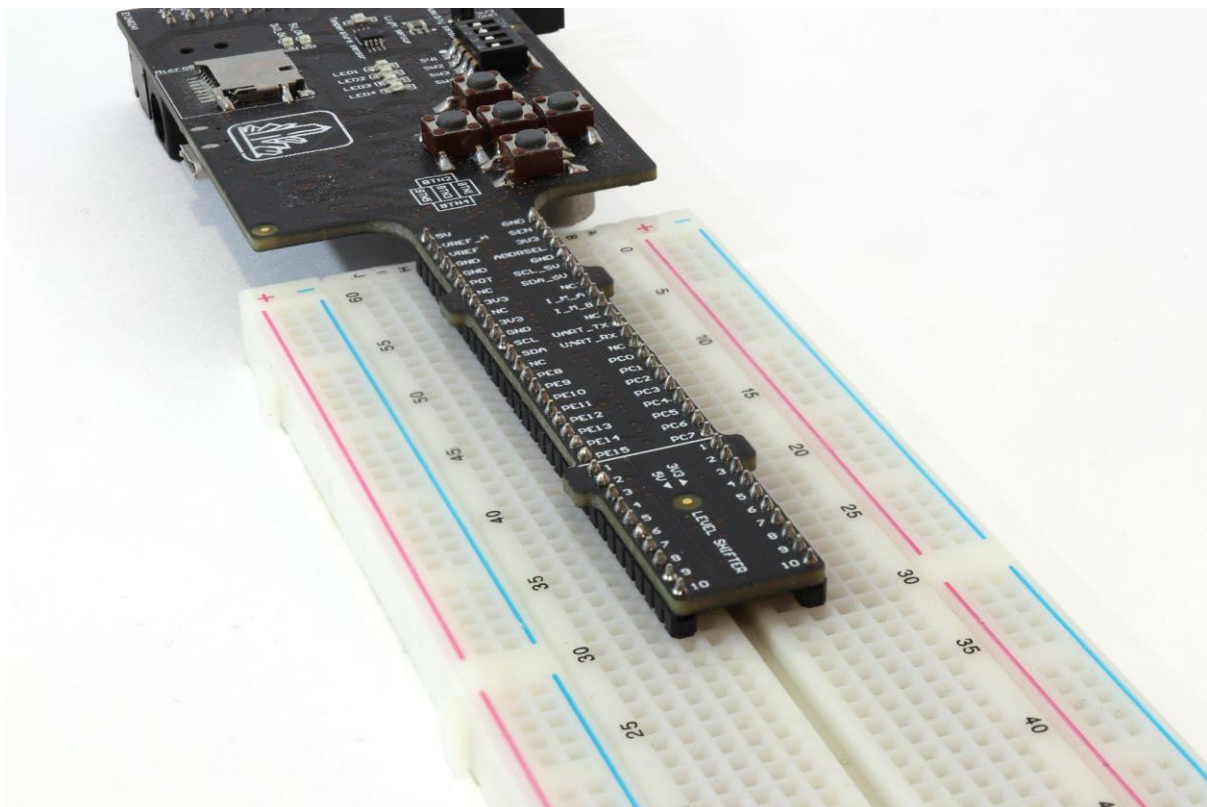
Item	Symbol	Condition	Min	Typ	Max	Unit
Supply Voltage for Logic	VCC	—	2.8	3.3	5.2	V
Input High Volt.	VIH	—	$0.8 \times VCC$	—	VCC	V
Input Low Volt.	VIL	—	0	—	$0.2 \times VCC$	V
Output High Volt.	VOH	—	$0.9 \times VCC$	—	VCC	V
Output Low Volt.	VOL	—	0	—	$0.1 \times VCC$	V
50% Check Board operating Current	ICC	Vcc=3.3V	—	14.0	22.0	mA

18. ábra: Általunk használt OLED kijelző különböző feszültségszintjei, fogyasztása

Forrás: Midas Displays

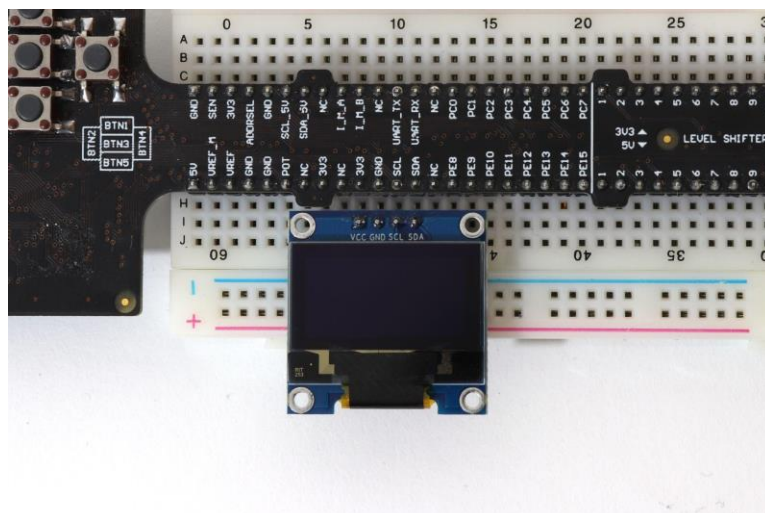
<https://www.midasdisplays.com/getattachment/785afe2f-1212-4e1f-8ab9-530d4a931614/specification>

Mivel a periféria áramfelvétele elég alacsony (ha a pixelek fele világít akkor is csak 22 mA), ezért külső tápforrás alkalmazása nem szükséges (vagyis elegendő a debuggeren keresztül érkező 3,3 V). Ezeknek a tudatában első lépésként illesszük a fejlesztőkártyát a breadboard-ra!



19. ábra: Fejlesztőpanel csatlakoztatása a breadboardhoz

A második lépésben csatlakoztassuk a kijelzőt a breadboard megfelelő pontjain keresztül a fejlesztőkártyához! Nem véletlenül kerültek ezek a lábak ilyen sorrendben egymás mellé, a hardvertervezők külön figyeltek a kijelző egyszerű illesztésére.



20. ábra: OLED kijelző megfelelő csatlakoztatása

Legvégül a debugger-re van szükségünk, melyet a már jól ismert módon tudtok a kapcsoláshoz hozzáadni.

A hardveres feladatokkal ezennel meg is volnánk, folytassuk hát a szoftverfejlesztéshez szükséges adatok begyűjtésével! Mivel grafikus kijelzőről beszélünk, ezért tudnunk kell a méretét, melyet az adatlap rögtön az elején tisztáz is.

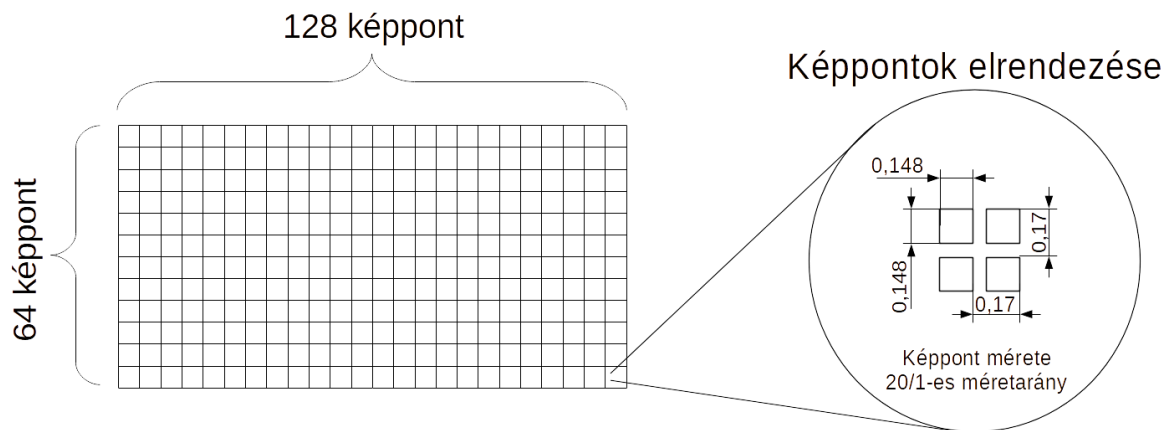
Display Features		 RoHS compliant	
Resolution	128 x 64		
Appearance	Blue on Black		
Logic Voltage	2.8V		
Interface	Multi		
Module Size	26.70 x 19.26 x 1.45mm		
Operating Temperature	-40°C ~ +80°C		
Construction	COT	Box Quantity	Weight / Display
		---	---

21. ábra: OLED kijelző paraméterei

Forrás: Midas Displays

<https://www.midasdisplays.com/getattachment/785afe2f-1212-4e1f-8ab9-530d4a931614/specification>

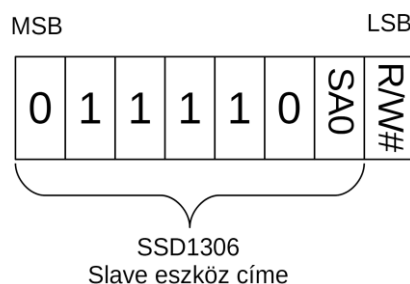
Ez azt jelenti, hogy 128 x 64 darab képponttal dolgozhatunk, melyet a következőképpen kell elképzelni.



22. ábra: OLED kijelző képpontjainak felépítése

Ez a mi „vásznunk”, erre lehet festeni. Esetünkben alaphoz ez egy fekete háttér, melyre világos pontokat tudunk rajzolni. Azért világos, mivel nagyon sok szín jelentheti a világos színt, például sárga, vörös vagy akár a mi esetünkben a kék. Ha elég ügyesen csináljuk ezt a festést, akkor a pontokból betűk, számok, alakzatok, képek lesznek. Természetesen lehetőségünk van a színek invertálására (amikor a világos háttérre feketével rajzolunk), ezt majd meg is nézzük, hogyan tehetjük meg.

Még mielőtt az adatlapokat félretennénk, egy dolgot kell kideríteni, mégpedig, hogy az I²C buszon milyen címen várja az utasításokat a megjelenítő. Ezt az adatot az Oled_kijelző_datasheet.pdf dokumentum 8.1.5 MCU I²C Interface részénél találhatjuk meg. A fejezetben láthatunk egy ábrát, ami jól szemlélteti a cím összetételét:



23. ábra: OLED kijelző I²C címe

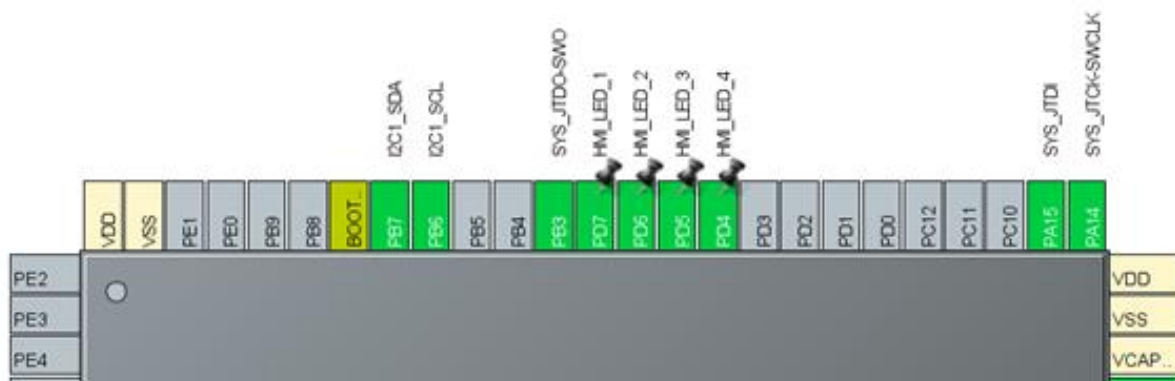
A slave address a 0b011110 bitekből és az SA0 értékéből tevődik össze. Utóbbi esetünkben legyen nulla (mivel a modulon ez a láb egy 0 Ohm-os ellenállással a földre van kötve), így a cím a bináris 0b0111100, ami hexadecimálisan 0x3C. Ez a cím megtalálható az egyik forrásfájlban, amit majd később látni fogunk.

SZOFTVER

Projekt létrehozása

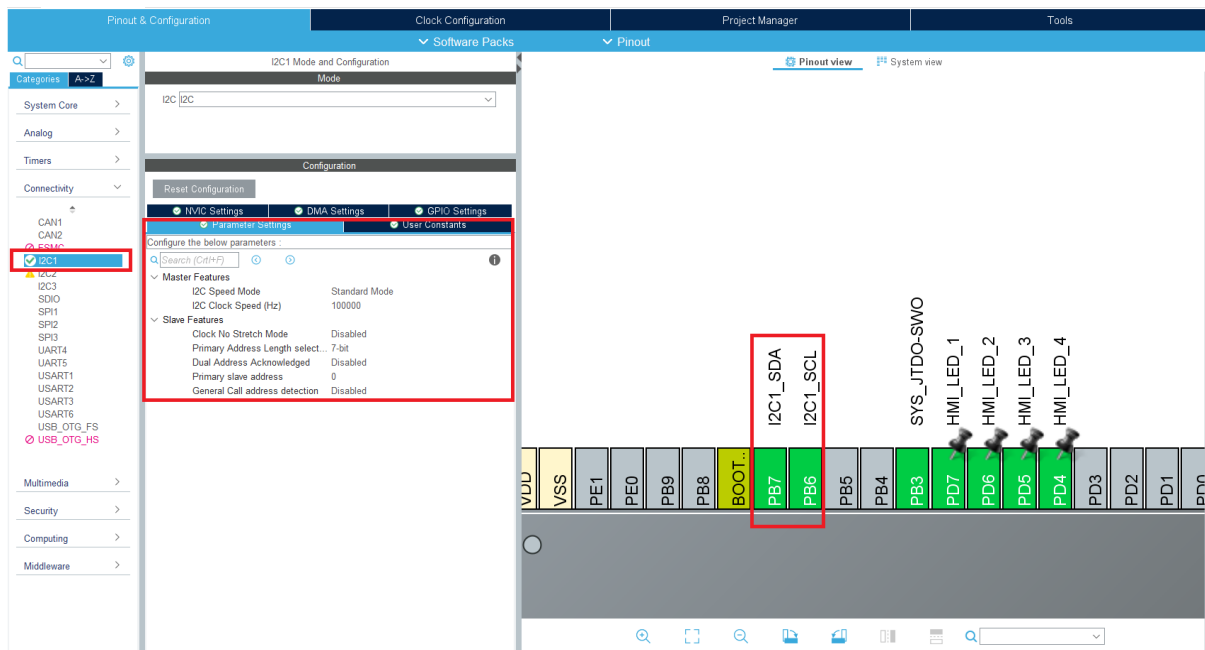
Kezdjük is hozzá a szoftveres projektünkhöz, melyet egy előre létrehozott programon keresztül mutatunk be nektek. Nézzük át együtt, értelmezzük, hogy a következő alkalommal magatoktól is egyszerűen össze tudjatok rakni egy kijelzőt tartalmazó projektet!

A korábbiakhoz hasonlóan készítsünk egy új STM32 Projektet a meglévő STM32CubeMX konfigurációs fájl felhasználásával. A „Pinout & Configuration” fülön láthatjuk a mikrokontroller lábkiosztását és beállításait.



24. ábra: Mikrovezérlő lábkiosztása a példakódban

Az I²C SDA és SCL lábait a PB7-es és PB6-os pinekre kötöttük a hardvernek megfelelően, mely az I2C1-es perifériához tartozik. A kommunikációs protokoll beállítását is előcsalhatjuk az I²C menüpont megnyitásával, mely a következő:



25. ábra: I²C beállításai

A többi HMI interfész is a projekt részét képezi, melyek egy része már ismerős lehet számodra:

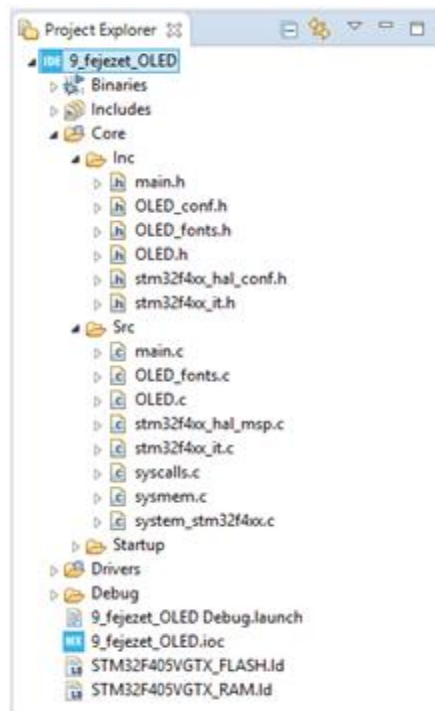
Elnevezés	Számozás	Beállítás	Megjegyzés
HMI_LED_x	1..4	GPIO_Output	Korábbi tananyagrészekben már használt periféria.
HMI_BTN_x	1..5	GPIO_Input	Korábbi tananyagrészekben már használt periféria.
HMI_SW_x	1..4	GPIO_Input	A kapcsoló még ismeretlen számodra, de nagyon hasonlít a használata a gomb kezelésére, állapota ugyanazzal az egy függvényhívással beolvasható.



26. ábra: Példa programhoz tartozó HMI interfészek

Most egyelőre ne változtassunk a kódon, maradjunk meg az eredeti konfigurációnál.

Navigáljunk el a „Project Explorer” nézetbe és vegyük sorra az újdonságokat, az eddigi ismereteinkhez képest.



27. ábra: Project Explorer felépítése a példakódban

A Core mappa Inc és Src almappjaiban találunk OLED kijelzővel kapcsolatos fájlokat, melyeket most közösen át is nézünk. Kezdjük az OLED_conf.h állománnyal, amely a kijelzőnk használatához szükséges felhasználói konfigurációt tartalmazza. A kijelzőt ugye I²C buszon küldött parancsokkal tudjuk vezérelni, amihez a már ismert HAL_I2C_Mem_Write() függvényt érdemes használnunk. Ennek a függvénynek két hardverfüggő paramétere van:

- I2C_HandleTypeDef, ami egy mutató a bekonfigurált I²C struktúrára állítva
- 16 bites cím, ami a céleszköz címe

Annak érdekében, hogy modulárisa tegyük a kódunkat, ezt a két paramétert a következő két makróval definiáltuk:

```
// I2C beállítás
#define OLED_I2C_PORT          hi2c1
#define OLED_I2C_ADDR          (0x3C << 1)
```

A portnak megadott pointer a generált I²C struktúrára mutat, ami a main.c fájlban található. Címként az adatlapban kikeresett 0x3C-t adtuk meg, amit egy bittel ugye el kell tolni a küldés során.

Az OLED library

Miért lesz ez ettől moduláris? A célunk szoftverfejlesztőként az átlátható és újrahasznosítható kódolás, ami elősegíti a szoftvermodulok (vagy szoftverkomponensek) könnyű cserélhetőségét, egyszerű

integrálhatóságát. Ha a szoftver többi részében az OLED_I2C_PORT és az OLED_I2C_ADDR makrókat használjuk, akkor egy esetleges hardveres változást gyorsan le tudunk kezelni minimális programozással. Ha például a mikrokontroller másik lábain lévő I²C-re átkerül a kijelzőnk, akkor elég az 5. sorban lévő makró értékét módosítani és az OLED driver kompatibilis lesz az új konfigurációval. Másik példa, ha OLED kijelzőt cserélünk és megváltozik a címe, akkor elég a 6. sorban lévő makró frissíteni, és máris ismerősként köszönhetjük a buszon.

Kijelző specifikus paraméter még a megjelenítő magassága és szélessége. Ezekből kiszámítjuk azt a tömb méretet, amivel leképezhetjük a képernyő pontjait. Egy pixelt egy bittel tudunk leírni, így a teljes kijelző mérete 8 192 biten, vagyis 1 024 bájtban ábrázolható.

```
// OLED kijelző magassága és szélessége pixelben
#define OLED_HEIGHT          64
#define OLED_WIDTH           128

// Buffer méret számítás bájtban
#define OLED_BUFFER_SIZE     OLED_WIDTH * OLED_HEIGHT / 8
```

A konfigurációs fájlunkban láthatjuk továbbá, hogy lehetőségünk van a kijelző vertikális és horizontális tükrözésére, valamint a színek invertálására. Ezeket az opciókat úgy tudjuk aktiválni, hogy a kommentezést kivesszük a define-ok elől (kitöröljük a “//” jeleket a sor elejéről). A define-ok itt úgynevezett fordítási direktívák, ami azt jelenti, hogy a forráskód egy része ezektől a kapcsolóktól függően fog lefordulni. Hamarosan láthatunk példát a használatukra.

```
// Kijelző tükrözés, amennyiben szükséges
// #define OLED_MIRROR_VERT
// #define OLED_MIRROR_HORIZ

// Inverz színbeállítás, amennyiben szükséges
// #define OLED_INVERSE_COLOR
```

1.1.1.1 A betűk ábrázolásmódja

Mivel grafikus a kijelzőnk, ezért a képpontokat egyesével kell vezérelnünk, hogy fekete vagy világos színűek legyenek. Ha szöveget szeretnénk kiírni a megjelenítőre, akkor szintén egyesével meg kell mondani, hogy adott karakter esetén melyik képpontokat kell “befesteni”. Ehhez létre kell hoznunk előre definiált karakterkészleteket, melyek tartalmazzák a betűk vizualizációját. Ha több betűméretet vagy betűtípust szeretnénk használni az alkalmazásunkban, akkor mindegyik mérethez és típushoz külön-külön karakterkészletet kell alkotnunk. Jelenleg a kódunk 4 betűméretet támogat: a legkisebb 6 képpont széles és 8 képpont magas, a legnagyobb 16 képpont széles és 26 képpont magas.

Forrásfájlunk utolsó kódrészletében azt állíthatjuk be, hogy a négy karakterkészletből melyek legyenek elérhetőek a szoftverünkben. Példánkban ezek közül mindegyik betűméret elérhető. Ha valamelyikre nincs szükségünk és szeretnénk a kód méretét csökkenteni, akkor kommentezzük ki az adott sort.

```
// Csak a szükséges betűtípusokat tartalmazza

#define OLED_INCLUDE_FONT_6x8           // 6 pixel széles, 8 pixel magas
#define OLED_INCLUDE_FONT_7x10          // 7 pixel széles, 10 pixel magas
#define OLED_INCLUDE_FONT_11x18         // 11 pixel széles, 18 pixel magas
#define OLED_INCLUDE_FONT_16x26         // 16 pixel széles, 26 pixel magas
```

Ha már a betűknél járunk, vizsgáljuk meg, hogyan néznek ki a különböző karakterkészletek implementációi! Az OLED_fonts.h állománnyal kezdjünk, melynek legelején láthatjuk beillesztve (#include) az OLED_conf.h állományunkat. Ezt követően egy FontDef nevű struktúradefiníció látható.

```
typedef struct
{
    const uint8_t FontWidth;    // Betűtípus szélesség pixelben
    uint8_t FontHeight;        // Betűtípus magasság pixelben
    const uint16_t *data;       // Mutató a betűtípusok adatait tartalmazó tömbre
} FontDef;
```

Ezzel a struktúrával tudjuk leírni a karaktereinket, melynek három eleme van: betűmagasság, betűszélesség, és a betűkészletre mutató pointer. Az OLED_fonts.c fájlra ugorva szemrevételezzük a struktúránk felhasználását, amit a 403. sortól kezdődően találunk.

```
#ifndef OLED_INCLUDE_FONT_6x8
FontDef Font_6x8 = {6,8,Font6x8};
#endif
```

A fordítási direktívával ellátott blokkot a következőképpen kell értelmezni: Az #ifndef - #endif preprocesszor direktívák közötti kódrészlet akkor fordítódik le a végső programunkba, ha az #ifndef-et követő kifejezés korábban már definiálásra került, vagy logikai értéke 1 (igaz). Mivel az OLED_INCLUDE_FONT_6x8-at az OLED_conf.h fájlban definiáltuk, és azt include-oltuk ebbe a fájlba közvetve az OLED_fonts.h-n keresztül, ezért a 404. sor bele fog fordulni a programunkba. Ez a sor nem csinál mást, mint létrehoz egy Font_6x8 nevű struktúrát, melynek kezdőértéket ad. A szélességet és magasságot az elnevezésnek megfelelően állítjuk be, illetve megadjuk azt a mutatót, ami a betűkészletünkhöz vezet. A kurzort állítsuk a Font6x8-ra a 404. sorban (kijelöljük vagy belekattintunk) és nyomjuk meg az F3-as billentyűt, hogy átugorjunk a Font6x8 deklarációjára, melynek csak egy részét mutatjuk itt meg:

```
#include "OLED_fonts.h"

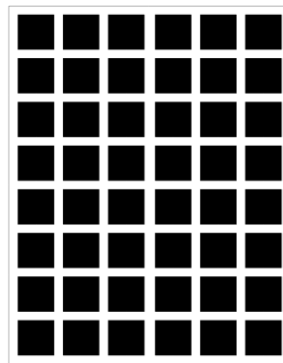
#ifndef OLED_INCLUDE_FONT_6x8
static const uint16_t Font6x8 [] = {
0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, // sp
```

```
0x2000, 0x2000, 0x2000, 0x2000, 0x2000, 0x0000, 0x2000, 0x0000, // !
0x5000, 0x5000, 0x5000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, // "
0x5000, 0x5000, 0xf800, 0x5000, 0xf800, 0x5000, 0x5000, 0x0000, // #
0x2000, 0x7800, 0xa000, 0x7000, 0x2800, 0xf000, 0x2000, 0x0000, // $
0xc000, 0xc800, 0x1000, 0x2000, 0x4000, 0x9800, 0x1800, 0x0000, // %
```

Mi az, amit látunk?

1. A már ismerős preprocesszor direktíva értelemszerűen itt is kell
2. A Font6x8 nevű betűkészlet egy tömb, melynek elemei 16 bites előjel nélküli egészek
3. A betűkészlet állandó, tehát deklarálható konstansként
4. Statikus, mert nem szeretnénk ezt a tömböt más forrásfájlból elérni

Megvizsgálva a tömbünk elemeit láthatjuk, hogy minden sorban 8 darab adat foglal helyet, melyek együtt írnak le egy karaktert. Vizualizáljuk a könnyebb megértéshez:



28. ábra: 6x8 méretű betűtípus felépítése

Minden betűnk szélessége 6 képpontnyi, magassága 8 képpontnyi (6 oszlop, 8 sor). A 8 darab adat a kódunk 1 sorában a karakterünk 8 sorát írja le fentről lefelé.

Példaként válasszuk a # karaktert, ami a kódunk 8. sora:

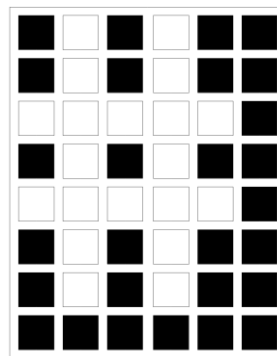
```
0x5000, 0x5000, 0xf800, 0x5000, 0xf800, 0x5000, 0x5000, 0x0000, // #
```

Az első elem az első sort, a második elem a második sort jelenti és így tovább:

■ ■ ■ ■ ■ ■	0x5000, 0b010100
■ ■ ■ ■ ■ ■	0x5000, 0b010100
■ ■ ■ ■ ■ ■	0xf800, 0b111110
■ ■ ■ ■ ■ ■	0x5000, 0b010100
■ ■ ■ ■ ■ ■	0xF800, 0b111110
■ ■ ■ ■ ■ ■	0x5000, 0b010100
■ ■ ■ ■ ■ ■	0x5000, 0b010100
■ ■ ■ ■ ■ ■	0x0000, 0b000000

29. ábra: Egyes sorokhoz tartozó adatok

Az adatok 16 bitesek, viszont csak a felső 6 bitre van szükségünk, a harmadik oszlopban ezért csak a felső 6 bit értéke lett megjelenítve. Ezeket a bináris értékeket jelenítsük meg a bal oldali „kijelzőn” az OLED technológiának megfelelően, a 0 jelentse a fekete színt, az 1-es pedig a fehér színt.



30. ábra: Az előző képen ismertetett adatokhoz tartozó kijelzés

Most már tudjuk, hogy mit jelentenek a sorok, de mi alapján rendszereztük a karakterkészletet reprezentáló tömböt? A karakterek sorrendje az ASCII kódtábla alapján lett felépítve, annyi különbséggel, hogy az első 32 karaktert és az utolsó karaktert nem vettük fel a karakterkészletbe.

The ASCII code

American Standard Code for Information Interchange

ASCII control characters				ASCII printable characters												Extended ASCII characters														
DEC	HEX	Simbolo ASCII		DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo
00	00h	NULL	(carácter nulo)	32	20h	espacio	64	40h	@	96	60h	`	128	80h	Ç	160	A0h	à	192	C0h	À	224	E0h	Ó	256	F0h	ou	288	H0h	ou
01	01h	SOH	(inicio encabezado)	33	21h	!	65	41h	A	97	61h	a	129	81h	ü	161	A1h	á	193	C1h	Á	225	E1h	ô	257	F1h	ov	289	H1h	ou
02	02h	STX	(inicio texto)	34	22h	"	66	42h	B	98	62h	b	130	82h	é	162	A2h	â	194	C2h	Â	226	E2h	ó	258	F2h	ow	290	H2h	ou
03	03h	ETX	(fin de texto)	35	23h	#	67	43h	C	99	63h	c	131	83h	ä	163	A3h	ã	195	C3h	Ã	227	E3h	ô	259	F3h	ox	291	H3h	ou
04	04h	EOT	(fin transmisión)	36	24h	\$	68	44h	D	100	64h	d	132	84h	å	164	A4h	ä	196	C4h	Ä	228	E4h	ó	260	F4h	oy	292	H4h	ou
05	05h	ENQ	(enquiry)	37	25h	%	69	45h	E	101	65h	e	133	85h	æ	165	A5h	å	197	C5h	Å	229	E5h	ô	261	F5h	oz	293	H5h	ou
06	06h	ACK	(acknowledgement)	38	26h	&	70	46h	F	102	66h	f	134	86h	ä	166	A6h	ä	198	C6h	Ä	230	E6h	ó	262	F6h	ow	294	H6h	ou
07	07h	BEL	(timbre)	39	27h	'	71	47h	G	103	67h	g	135	87h	ç	167	A7h	ç	199	C7h	Ç	231	E7h	ô	263	F7h	ox	295	H7h	ou
08	08h	BS	(retroceso)	40	28h	(	72	48h	H	104	68h	h	136	88h	è	168	A8h	è	200	C8h	È	232	E8h	ó	264	F8h	ow	296	H8h	ou
09	09h	HT	(tab horizontal)	41	29h	)	73	49h	I	105	69h	i	137	89h	é	169	A9h	é	201	C9h	É	233	E9h	ô	265	F9h	ox	297	H9h	ou
10	0Ah	LF	(salto de línea)	42	2Ah	*	74	4Ah	J	106	6Ah	j	138	8Ah	è	170	AAh	è	202	CAh	Ê	234	EAh	ó	266	FAh	ow	298	HAh	ou
11	0Bh	VT	(tab vertical)	43	2Bh	+	75	4Bh	K	107	6Bh	k	139	8Bh	ï	171	ABh	ï	203	CBh	Ë	235	EBh	ô	267	Fbh	ow	299	Hbh	ou
12	0Ch	FF	(form feed)	44	2Ch	,	76	4Ch	L	108	6Ch	l	140	8Ch	ï	172	ACH	ï	204	CAh	Ë	236	ECh	ó	268	Fch	ow	300	Hch	ou
13	0Dh	CR	(retorno de carro)	45	2Dh	-	77	4Dh	M	109	6Dh	m	141	8Dh	ï	173	ADh	ï	205	CDh	Ë	237	EDh	ô	269	Fdh	ow	301	Hdh	ou
14	0Eh	SO	(shift Out)	46	2Eh	.	78	4Eh	N	110	6Eh	n	142	8Eh	Ä	174	AEh	Ä	206	CEh	Ë	238	Eeh	ó	270	Feh	ow	302	Heh	ou
15	0Fh	SI	(shift in)	47	2Fh	/	79	4Fh	O	111	6Fh	o	143	8Fh	Ä	175	AFh	Ä	207	CFh	Ë	239	Efh	ô	271	Ffh	ow	303	Hfh	ou
16	10h	DLE	(data link escape)	48	30h	0	80	50h	P	112	70h	p	144	90h	Ë	176	B0h	Ë	208	D0h	Ð	240	F0h	ó	272	Fgh	ow	304	Hgh	ou
17	11h	DC1	(device control 1)	49	31h	1	81	51h	Q	113	71h	q	145	91h	æ	177	B1h	æ	209	D1h	Ð	241	F1h	ô	273	Fgh	ow	305	Hgh	ou
18	12h	DC2	(device control 2)	50	32h	2	82	52h	R	114	72h	r	146	92h	Æ	178	B2h	Æ	210	D2h	Ð	242	F2h	ó	274	Fgh	ow	306	Hgh	ou
19	13h	DC3	(device control 3)	51	33h	3	83	53h	S	115	73h	s	147	93h	ö	179	B3h	ö	211	D3h	Ð	243	F3h	ô	275	Fgh	ow	307	Hgh	ou
20	14h	DC4	(device control 4)	52	34h	4	84	54h	T	116	74h	t	148	94h	ö	180	B4h	ö	212	D4h	Ð	244	F4h	ó	276	Fgh	ow	308	Hgh	ou
21	15h	NAK	(negative acknowle.)	53	35h	5	85	55h	U	117	75h	u	149	95h	ö	181	B5h	ö	213	D5h	Ð	245	F5h	ô	277	Fgh	ow	309	Hgh	ou
22	16h	SYN	(synchronous idle)	54	36h	6	86	56h	V	118	76h	v	150	96h	ü	182	B6h	ü	214	D6h	Ð	246	F6h	ó	278	Fgh	ow	310	Hgh	ou
23	17h	ETB	(end of trans. block)	55	37h	7	87	57h	W	119	77h	w	151	97h	ü	183	B7h	ü	215	D7h	Ð	247	F7h	ô	279	Fgh	ow	311	Hgh	ou
24	18h	CAN	(cancel)	56	38h	8	88	58h	X	120	78h	x	152	98h	y	184	B8h	ÿ	216	D8h	Ð	248	F8h	ó	280	Fgh	ow	312	Hgh	ou
25	19h	EM	(end of medium)	57	39h	9	89	59h	Y	121	79h	y	153	99h	ÿ	185	B9h	ÿ	217	D9h	Ð	249	F9h	ô	281	Fgh	ow	313	Hgh	ou
26	1Ah	SUB	(substitute)	58	3Ah	:	90	5Ah	Z	122	7Ah	z	154	9Ah	ÿ	186	BAh	ÿ	218	DAh	Ð	250	FAh	ó	282	Fgh	ow	314	Hgh	ou
27	1Bh	ESC	(escape)	59	3Bh	;	91	5Bh	[	123	7Bh	{	155	9Bh	£	187	BBh	£	219	DBh	Ð	251	FBh	ô	283	Fgh	ow	315	Hgh	ou
28	1Ch	FS	(file separator)	60	3Ch	<	92	5Ch	\	124	7Ch		156	9Ch	£	188	BCb	£	220	DCh	£	252	FCh	ó	284	Fgh	ow	316	Hgh	ou
29	1Dh	GS	(group separator)	61	3Dh	=	93	5Dh	]	125	7Dh	}	157	9Dh	Ø	189	BDh	Ø	221	DDh	£	253	FDh	ô	285	Fgh	ow	317	Hgh	ou
30	1Eh	RS	(record separator)	62	3Eh	>	94	5Eh	^	126	7Eh	~	158	9Eh	x	190	BEh	x	222	DEh	£	254	FEh	ó	286	Fgh	ow	318	Hgh	ou
31	1Fh	US	(unit separator)	63	3Fh	?	95	5Fh	_				159	9Fh	f	191	BFh	f	223	DFh	£	255	FFh	ô	287	Fgh	ow	319	Hgh	ou
127	20h	DEL	(delete)																											

theASCIIcode.com.ar

31. ábra: ASCII kódtábla

Forrás: GifTagger [CC BY-SA 3.0 (<https://creativecommons.org/licenses/by-sa/3.0/>)]
<https://commons.wikimedia.org/wiki/File:Ascii-codes-table.png>

Felmerülhet még bennünk az a kérdés, hogy miért 16 bites tömböt használunk, ha csak 6 bit hasznos számunkra? Ha megnézzük a karakterkészleteinket, akkor a legnagyobb méretű betű az 16x26 pixeles. Annak érdekében, hogy kompatibilisek legyenek a karakterkészleteink, olyan adattípust választottunk, melyen minden betűméret ábrázolható.

Térjünk vissza még néhány szó erejéig az OLED_fonts.h állományunkra, annak is a végére.

```
#ifndef OLED_INCLUDE_FONT_6x8
extern FontDef Font_6x8;
#endif
```

Az extern kulcsszó azt jelzi a fordítónak, hogy van valahol egy Font_6x8 nevű, FontDef típusú változó, ami elő fog kerülni linkelés során, ezért fordítsa le nekünk. Gyakorlatilag kiejánljuk használatra a Font_6x8-at azok számára, akik beillesztik a fejlécfájlt.

Két állományunk maradt, amelyek közvetlenül kapcsolódnak az OLED kijelzőhöz. Ezekben a fájlokban vannak implementálva azok a funkciók, melyek alkalmazásával életre kelthetjük a kijelzőnket. Elemezzük közösen az egyes kódrészleteket, kezdjük ismét a fejlécfájlnak belsejével az eddigiekhez hasonlóan!

Első lépésben include-olva van az összes eddigi munkánk az OLED_fonts.h formájában. Megtalálható a korábban definiált I2C_PORT deklarációja, hogy a fordítót útba igazítsuk. Ezeket követi néhány felsorolás típus, vagy angol nevén enum, melyek a kódunk érthetőségét növelik számunkra.

A létrehozott enum-ok a következők:

- OLED_Color - szín
 - BLACK - ez jelzi a fekete színt
 - WHITE - ez jelzi a világos színt
- OLED_Switch - kapcsoló
 - OFF - kikapcsolás jelentésével bír
 - ON - bekapcsolás jelentésével bír

A felhasználási helyen láthatjuk majd, hogy milyen egyszerűvé válik a kódolás és a kód analízise a felsorolt típusoknak köszönhetően.

A következő rész két struktúradefiníció:

```
// Az OLED kijelző tulajdonságainak tárolására szolgáló struktúra
typedef struct
{
    uint16_t CurrentX;           // Aktuális X pozíció
    uint16_t CurrentY;           // Aktuális Y pozíció
    uint8_t Inverted;            // Invertálás
    uint8_t Initialized;         // Inicializálva
    uint8_t DisplayOn;           // Bekapcsolás
} OLED_t;

// Struktúra a koordináták tárolására
typedef struct
{
    uint8_t x;
    uint8_t y;
} OLED_Vertex;
```

Az első struktúrában az OLED kijelző olyan tulajdonságait találjuk, melyek elengedhetetlenek az alapvető és rendeltetésszerű működtetéséhez. Felülről haladva az első két elem a kijelző virtuális kurzorát reprezentálja. Ezekkel tudunk megadni egy pontot a kijelzőn, például, hogy honnan kezdve szeretnénk szöveget megjeleníteni. A maradék elemeink a struktúrában különböző beállítások számontartására szolgálnak: nyilvántarthatjuk, hogy aktív-e a megjelenítő invertálásának funkciója, hogy sikeresen végbement-e az inicializáció folyamata, illetve, hogy be van-e kapcsolva a kijelző. Sőt, mondhatjuk, hogy ezeket kötelező nyilvántartani.

Az OLED_Verx struktúra koordinátákat fog össze a könnyebb kezelhetőség érdekében.

A fejlécfájl végén azoknak a függvényeknek a deklarációját találjuk, amelyeket szeretnénk elérhetővé tenni a modulon kívül is. Modulnak a logikailag egybetartozó programrészt hívjuk jelen esetben. Az itt felsorolt függvények megvalósításait a forrásfájlban találjuk meg, lépünk is tovább hát a tanulmányozásban.

A megszokott módon az include-okkal kezdünk, ezeket követik a makróként definiált konstansok, majd néhány változódefiníció, legvégül pedig a függvényimplementációk. Két fontos változóval találkozhattunk:

- SSD1306 - ebben az struktúrában tárolhatjuk a kijelző aktuális állapotait, beállításait.
- OLED_Buffer - ebben tudjuk a kijelzőnek lokális másolatát tartani. Ha valamit ki szeretnénk rajzolni, akkor azt először ebben a leképzésben elvégezzük, majd ezt küldjük el a kijelzőnek. (A kijelző szintjén ezt úgy tudjuk elképzelni, hogy a neki küldött adatokat elmenti egy RAM memóriába, amit aztán bitről bitre megjelenít.)

Következhet a függvények szemrevételezése az alábbi kérdésekre keresve a választ: mit csinálnak és hogyan működnek? Logikailag olyan sorrendben építettük fel a forrásfájlt, hogy az alapvető működéshez szükséges funkciók az elején találhatóak, amelyek elengedhetetlenek a kijelző működtetéséhez. Ilyen nélkülözhetetlen elemek a kommunikációs függvények, melyek használatával parancsokat vagy adatokat küldhetünk az OLED modul számára.

```
// Bájít írása a parancsregiszterbe
void OLED_WriteCommand(uint8_t byte)
{
    HAL_I2C_Mem_Write(&OLED_I2C_PORT, OLED_I2C_ADDR, 0x00, 1, &byte, 1,
HAL_MAX_DELAY);

    return;
}

// Adatok írása a parancsregiszterbe
void OLED_WriteData(uint8_t* buffer, size_t buff_size)
{
    HAL_I2C_Mem_Write(&OLED_I2C_PORT, OLED_I2C_ADDR, 0x40, 1, buffer, buff_size,
HAL_MAX_DELAY);

    return;
}
```

Mindkét esetben az I²C kommunikációhoz szükséges HAL_I2C_Mem_Write() függvényt használjuk, csupán annak bemeneti paramétereit változtatjuk. Portként és címként a korábban erre a célra definiált makrókat adjuk meg (OLED_I2C_PORT, OLED_I2C_ADDR). A harmadik paraméter alapján tesz

különbséget parancs és adat között az OLED kijelző. Ehhez az adatlapban a 8.2 Command Decoder résznél találunk információt.

Számunkra releváns paraméter még az ötödik és hatodik argumentum (uint8_t *pData és uint16_t Size). Előbbi a küldendő adatainkra mutató pointer, utóbbi pedig a küldendő adatok mérete. Parancs esetén ez 1 bájt nagyságú parancskód, melyet a kijelző adatlapjából tudunk kikeresni. Adat kiküldésekor a buffer címét és az adatcsomag méretét kell megadnunk.

Nézzük is meg a használatukat a gyakorlatban! Az OLED_SetDisplay() függvény arra szolgál, hogy a kijelzőt be- vagy kikapcsoljuk. Ennek törzsében láthatjuk, hogy a bemeneti paraméter alapján (ON vagy OFF) beállítjuk a kimeneti parancsunk kódját, frissítjük a kijelző állapotát a nyilvántartásra szolgáló struktúrában, majd kiküldjük a beállított parancsot.

```
//Kijelző ki- vagy bekapcsolása
void OLED_SetDisplay(OLED_Switch display_switch)
{
    uint8_t command;

    if (ON == display_switch)
    {
        // Bekapcsolás
        command = 0xAF;
        SSD1306.DisplayOn = 1;
    }
    else
    {
        // Kikapcsolás
        command = 0xAE;
        SSD1306.DisplayOn = 0;
    }

    // Kiválasztott parancs küldése a kijelzőnek
    OLED_WriteCommand(command);

    return;
}
```

Hogyan kereshetjük meg a parancsok kódjait? Az adatlapban a 9-10. fejezetek tartalmazzák a parancsokat, illetve a leírásaikat. Például a ki- és bekapcsolás utasítás a 9-1-es parancs táblázatban

(Table 9-1: Command Table), illetve a 10.1.12-es fejezetben van ismertetve (10.1.12 Set Display ON/OFF (AEh/AFh)).

Nagyon hasonló felépítésű a kijelző invertálását megvalósító függvényünk.

```
//Kijelző színinvertálásának ki- vagy bekapcsolása
void OLED_SetDisplayInverse(OLED_Switch inverse_switch)
{
    uint8_t command;

    if (ON == inverse_switch)
    {
        // Invertálás bekapcsolása
        command = 0xA7;
        SSD1306.Inverted = 1;
    }
    else
    {
        // Normál színek
        command = 0xA6;
        SSD1306.Inverted = 0;
    }

    // Kiválasztott parancs küldése a kijelzőnek
    OLED_WriteCommand(command);

    return;
}
```

Használat közben előfordulhat, hogy ellenőriznünk kell az invertáltság állapotát, amit egy lekérdező függvény segítségével tudunk megtenni. Miért nem vizsgáljuk az SSD1306 struktúrát a főprogramban? A válasz egyszerű, mert a főprogram elől el lett rejtve ez a struktúra, csupán ezeken a függvényeken keresztül lehet elérni. Ez a megoldás a modularizáltságot javítja, vagyis, ha az OLED könyvtár belseje megváltozik, akkor az a főprogramot nem fogja érinteni, feltéve, ha a modul interfészei változatlanok maradnak. A másik érv a struktúra elrejtése mellett, hogy a kijelzőt kezelő könyvtárnak kell számotartania a saját állapotait, az őt használók csak fogyasztói ezeknek az információknak.

```
//Invertálás állapotának kiolvasása
OLED_Switch OLED_GetDisplayInverse(void)
{
    return (SSD1306.Inverted);
}
```

Lehetőségünk van a megjelenítőnk kontrasztjának állítására.

```
// Kontraszt állítás
void OLED_SetContrast(const uint8_t value)
{
    // Kontrasztszabályozási regiszter kiválasztása
    OLED_WriteCommand(0x81);

    // Kiválasztott érték küldése a kijelzőnek
    OLED_WriteCommand(value);

    return;
}
```

A kijelző adott képpontjára állíthatjuk a kurzorunkat az OLED_SetCursor() függvény segítségével.

```
// Kurzor pozícióra állítás
void OLED_SetCursor(uint8_t x, uint8_t y)
{
    SSD1306.CurrentX = x; // x pozíció
    SSD1306.CurrentY = y; // y pozíció

    return;
}
```

Feketére vagy világosra tudjuk színezeni a teljes képernyőt az OLED_Fill() függvény segítségével. Ha bemeneti paraméternek a világos színt adjuk meg (WHITE), akkor a *color_value* minden egyes bitjét át kell állítanunk 1-esre, vagyis 0xFF-et kell értékül adnunk. Ha a bemeneti színünk a fekete (BLACK), akkor a *color_value* értékét nem kell módosítani, mert a kezdőértéke be van állítva 0-ra. Ezt követően már csak végig kell lépkedni a kimeneti bufferen és feltölteni a *color_value*-val.

```
// Kijelző adott színre történő kitöltése
void OLED_Fill(OLED_Color color)
{
    uint32_t i;
    uint8_t color_value = 0x00u;          // Fekete

    // Érték beállítás a kiválasztott színnek megfelelően
    if (WHITE == color)
    {
        color_value = 0xFF;
    }
    // Kiválasztott szín beállítása a bufferben
    for (i = 0; i < OLED_BUFFER_SIZE; i++)
    {
        OLED_Buffer[i] = color_value;
    }

    return;
}
```

Most már tudjuk, hogyan kell a kimeneti tömböt feltölteni egy adott színnel, de hogyan lehet ezt a kijelző részére továbbítani? Ezen célból áll rendelkezésünkre az `OLED_UpdateScreen()` névre keresztelt függvény, mely blokkonként kiküldi a buffer tartalmát. Egy blokkba 8 teljes kijelző sor van összefogva.

```
// Kijelző frissítése a bufferadatok alapján
void OLED_UpdateScreen(void)
{
    // Végigiterálás az egész bufferen
    for (uint8_t i = 0; i < (OLED_HEIGHT/8); i++)
    {
        // Aktuális RAM cím kiválasztása
        OLED_WriteCommand(0xB0 + i);
        OLED_WriteCommand(0x00);
        OLED_WriteCommand(0x10);

        // Értékek beírása
    }
}
```

```

        OLED_WriteData(&OLED_Buffer[OLED_WIDTH*i], OLED_WIDTH);
    }

    return;
}

```

1.1.1.2 Inicializáció

Közel minden perifériát az elindulást követően inicializálni kell, hogy a számunkra alkalmas beállításokkal tudjon működni. Esetünkben is így kell eljárni, meg kell hívni az OLED_Init() funkciót, amely elvégzi nekünk a kijelző indításakor szükséges konfigurálását. Részletesen nem szeretnénk analizálni a függvény belsejét, ugyanis ezt mi is az adatlapban közölt mintakód felhasználásával implementáltuk. Az *OLED_kijelző_datasheet.pdf* nevű adatlap 6. oldalán találtok egy rövid kódsort, némi kommentek mögé rejtett magyarázattal (init_SSD1306BZ() függvény). Hasonló megvalósításra bukkanhattok a saját kódunkban, minimális módosításokkal és kiegészítésekkel.

1.1.1.3 Alakzatok kirajzolása

Most mondhatjuk, hogy beértünk a célegyenesbe, ugyanis elérkeztünk a rajzolásra szolgáló függvényekhez. A legalapvetőbb dolog, amit tudnunk kell, az egy képpont átszínezése:

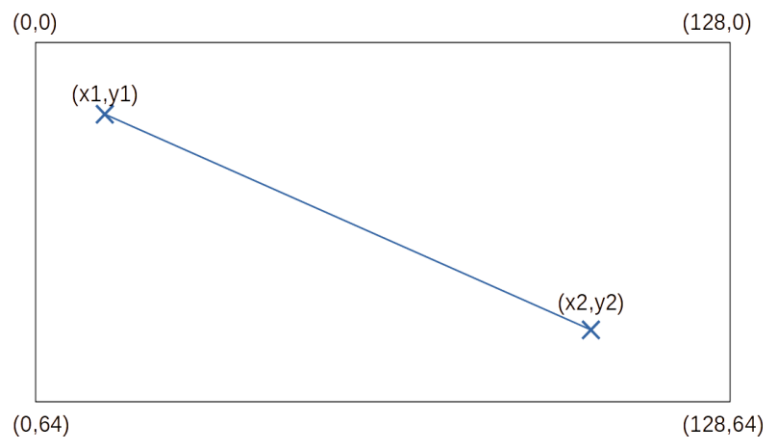
```

// Pixel rajzolás
void OLED_DrawPixel(uint8_t x, uint8_t y, OLED_Color color)
{
    // Kijelző méretének ellenőrzése
    if (OLED_WIDTH > x && OLED_HEIGHT > y)
    {
        // Pixel kirajzolása
        if (color == WHITE)
        {
            OLED_Buffer[x + (y / 8) * OLED_WIDTH] |= 1 << (y % 8);
        }
        else
        {
            OLED_Buffer[x + (y / 8) * OLED_WIDTH] &= ~(1 << (y % 8));
        }
    }
    return;
}

```

A függvénynek azt kell megmondanunk, hogy a megadott x és y koordinátákkal rendelkező pontot milyen színűre kívánjuk színeezni. Az `OLED_DrawPixel()` ellenőrzi, hogy a megadott képpont valós-e (rajta van a képernyőn), és amennyiben igen, akkor elvégzi a képpont beállítását a színnek megfelelően. Mivel egy nagy tömb adott elemének egyetlen bitjét szeretnénk csak módosítani, így bitműveleteket kell használnunk. Az 1-es értéket bitszintű VAGY művelettel, a 0-s értéket pedig bitszintű ÉS művelettel tudjuk beállítani, ezért kell két megoldás a két színhez.

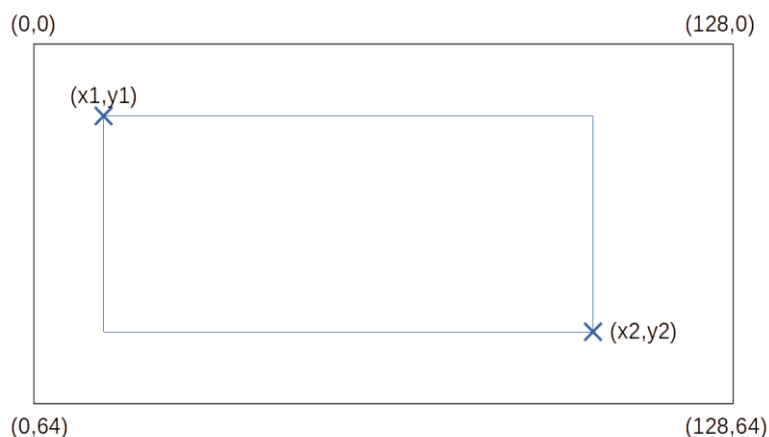
Ha pontot tudunk rajzolni, akkor jöhet a vonal, ami nem más, mint két pont között kifeszített egyenesre rajzolt pontok összessége.



32. ábra: `OLED_DrawLine()` működéséhez szükséges koordináták

Az `OLED_DrawLine()` függvény két pontot vár, pontonként két koordinátát, és természetesen a színt, amivel a vonalat szeretnénk kirajzolni. A vonal kirajzolásának algoritmusára nem térünk ki részletesen, de rá tudtok keresni az alábbi néven: Bresenham-féle¹ vonalas algoritmus.

Lépjünk tovább, mit is lehet vonalakból kirajzolni? Nekem a téglalap jutott eszembe, mint szabályos alakzat. Egy téglalap esetén elegendő 2 szemkötti csúcs koordinátáit megadni, hiszen a másik két csúcs pontjai ezekből származtathatóak. Ha pedig megvan mind a négy csúcs koordinátája, akkor használjuk fel a jól bevált vonalrajzoló függvényünket.



33. ábra: `OLED_DrawRectangle()` működéséhez szükséges koordináták

```
// Téglalap rajzolás

void OLED_DrawRectangle(uint8_t x1, uint8_t y1, uint8_t x2, uint8_t y2, OLED_Color
color)
{
    // Téglalap rajzolása vonalról vonalra
    OLED_DrawLine(x1, y1, x2, y1, color);
    OLED_DrawLine(x2, y1, x2, y2, color);
    OLED_DrawLine(x2, y2, x1, y2, color);
    OLED_DrawLine(x1, y2, x1, y1, color);

    return;
}
```

Mit lehet még vonalakból kirajzolni? Gyakorlatilag bármit. Annak érdekében, hogy a vonalakkal való ábrázolást támogassuk, létrehoztunk egy függvényt, amely képes a bemenetként megadott koordinátákat folytonos egyenes vonalakkal összekötni. Az OLED_DrawPolyline() függvény bemenete egy képpontokat (koordináta-párokat) tartalmazó tömb, a tömb mérete, és nem utolsósorban a kirajzolni kívánt vonalak színe.

```
// Vonallánc rajzolás

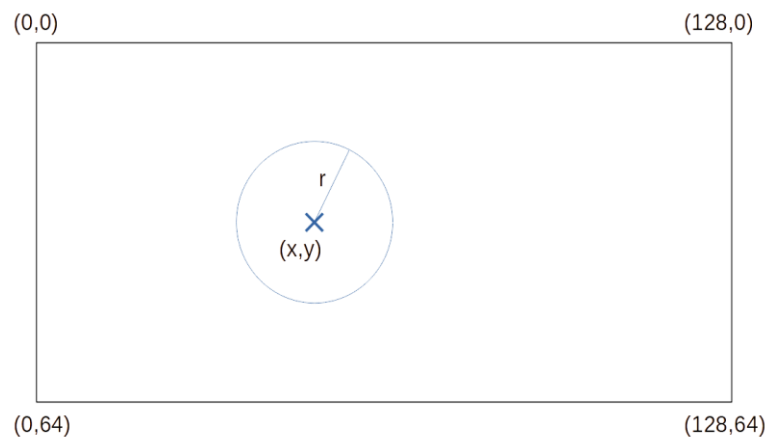
void OLED_DrawPolyline(const OLED_Vertex *vertex, uint16_t size, OLED_Color color)
{
    uint16_t i;
    // Tömb ellenőrzése
    if (0 != vertex)
    {
        // Végigiterálás a koordinátákat tartalmazó tömbön, vonalak rajzolása
        for (i = 1; i < size; i++)
        {
            OLED_DrawLine(vertex[i - 1].x, vertex[i - 1].y, vertex[i].x,
vertex[i].y, color);
        }
    }

    return;
}
```

Ennek a függvénynek a birtokában bármilyen sokszöget meg tudunk jeleníteni a kijelzőnkön.

Másik gyakran használt alakzat még a kör és a körív, melyeket szintén célszerűnek láttunk hozzáadni az eszközkészletünkhöz. A körünk megalkotásához Bresenham kör algoritmusát használtuk fel, melynek szintén utána tudtok nézni. Négy paramétert kell megadni az `OLED_DrawCircle()` függvénynek, melyek sorban a következők:

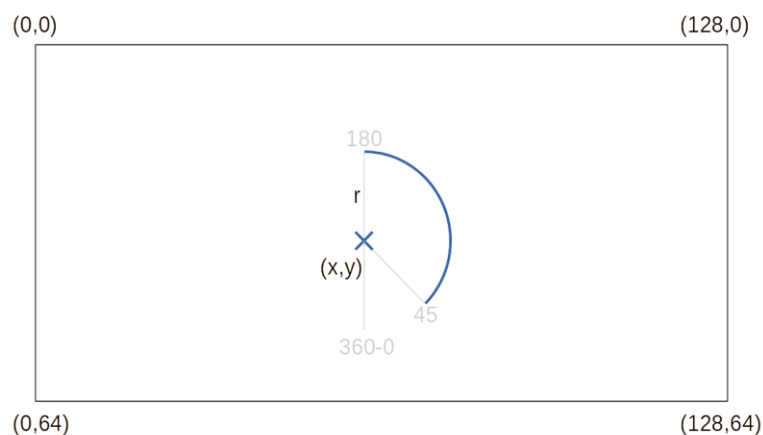
- a kör középpontjának x koordinátája
- a kör középpontjának y koordinátája
- a kör sugara
- a kör színe



34. ábra: `OLED_DrawCircle()` működéséhez szükséges paraméterek

A körívet rajzoló algoritmusra sem térek ki a tananyagrészen részletesen, csupán a paraméterek használatában segítenék:

- a körív középpontjának x koordinátája
- a körív középpontjának y koordinátája
- a körív sugara: r
- az a szög, amely megadja, hogy hol kezdődjön a körívünk: példánkban ez 45 fok (Vigyázat! Elsőre zavaró lehet, hogy ebben a könyvtárban a kör aljától számoljuk a szögek nagyságát!)
- az a szög, amely megadja, hogy hol végződjön a körívünk: példánkban ez 180 fok



35. ábra: `OLED_DrawArc()` működéséhez szükséges paraméterek

1.1.1.4 Szöveg és betűk kiírása

A rajzolásra használható függvényekkel ezennel végeztünk is. Még egy adósságunk van számotokra, az pedig a szöveg kiírása. A használható betűtípusokat már átvettük, most az a kérdés, ezek hogyan kerülnek rá a képernyőre. Kezdjük először is egy darab karakterrel és annak bufferbe írásával. Ehhez a feladathoz az OLED_WriteChar() függvényt hoztuk létre. A függvény hívásakor meg kell adni a kívánt karaktert, betűtípust és betűszínt.

```
// Karakter kirajzolás
char OLED_WriteChar(char ch, FontDef Font, OLED_Color color)
{
    uint8_t i, j;
    uint16_t font_part;
    char retVal = 0;

    // Karakter érvényességének vizsgálata
    if (32 <= ch && 126 >= ch)
    {
        // Maradék hely ellenőrzése az adott sorban
        if (OLED_WIDTH >= (SSD1306.CurrentX + Font.FontWidth) && OLED_HEIGHT >=
(SSD1306.CurrentY + Font.FontHeight))
        {
            // Paraméterként kapott betűtípus használata
            for (i = 0; i < Font.FontHeight; i++)
            {
                font_part = Font.data[(ch - 32) * Font.FontHeight + i];

                for (j = 0; j < Font.FontWidth; j++)
                {
                    if ((font_part << j) & 0x8000)
                    {
                        OLED_DrawPixel(SSD1306.CurrentX + j, (SSD1306.CurrentY +
i), (OLED_Color) color);
                    }
                    else
                    {
                        OLED_DrawPixel(SSD1306.CurrentX + j, (SSD1306.CurrentY +
i), (OLED_Color)!color);
                    }
                }
            }
        }
    }
    return retVal;
}
```

```

    }
}

// A betű által elfoglalt hely lefoglalása
SSD1306.CurrentX += Font.FontWidth;

// Kiírt karakter írása a visszatérési változóba ellenőrzés céljából
retVal = ch;
}

// Karakter vagy nulla (érvénytelen karakter és kevés hely esetén)
return (retVal);
}

```

A függvény első lépésként ellenőrzi, hogy az argumentumként kapott karakter érvényes-e, vagyis létezik-e az implementációja a kódunkban. Ha visszagondolunk korábbra, akkor mi a decimális 32-től a 126-os karakterig valósítottuk meg az ASCII karaktereket. Ha ebből az intervallumból kilóg a bemeneti karakterünk decimális értéke, akkor nem tudjuk kiírni ezt a karaktert, hibával (0 értékkel) térünk vissza a függvényből. Ellenkező esetben folytathatjuk a függvényünk végrehajtását egy újabb ellenőrzéssel, hogy a kívánt betűméret fizikailag kifer-e a képernyőre. Ehhez egyszerűen a kurzor aktuális koordinátáihoz hozzáadjuk a karakterünk szélességét és magasságát, ami, ha kisebb, mint a kijelző szélessége és magassága, akkor minden további nélkül elkezdhetjük a kiírást. A bufferbe írás két for ciklus segítségével van megvalósítva. A külső ciklus a betűnk magasságán iterál végig, míg a belső a karaktereink szélességén.

Szemléltessük egy példán keresztül: ki szeretnénk írni a “#” karaktert 7x10-es betűtípussal:

```

#ifdef OLED_INCLUDE_FONT_7x10

static const uint16_t Font7x10 [] = {
0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000, // sp
0x1000,0x1000,0x1000,0x1000,0x1000,0x1000,0x0000,0x1000,0x0000, // !
0x2800,0x2800,0x2800,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000, // "
0x2400,0x2400,0x7C00,0x2400,0x4800,0x7C00,0x4800,0x4800,0x0000, // #

```

7 képpont széles, 10 képpont magas karaktereket használunk, vagyis a külső for ciklusnak a 108. sor elemein kell végig iterálnia. A “#” decimális értéke 35, de mivel 32-től tartjuk nyilván a karaktereket, ezért konstans 32-t mindig ki kell vonnunk a dekódolt decimális értékből, majd megszorozni a betű magasságával. Ez azt jelenti, hogy ha a függvény bemenetére a “#” karakter kerül, akkor a font_part a

Font.data 30. elemével lesz egyenlő $[(35 - 32) * 10 = 30]$, ami az óhajtott 108. sor első eleme. A külső ciklusunk tehát ezen a tömbön lépked végig egyesével, míg el nem érjük az adott karakter utolsó elemét. A belső ciklusunk a kiválasztott tömbelemen halad (szakkifejezéssel élve iterál) végig bitről bitre, és azt vizsgálja, hogy ez 1 vagy 0, majd az eredménytől függően színezi be az adott pixelt az OLED_DrawPixel() függvény felhasználásával. Sikeres kiíratás esetén frissítjük a kurzorunk értékét és validációs célból a kiírt karaktert adjuk visszatérési értékként.

Újabb jó pont, hogy megtanultuk a betűk kiíratásának logikáját, innen már csak egy lépés a szöveg megjelenítése. A bemeneti szöveg karaktereire egyesével meghívjuk az OLED_WriteChar() függvényt mindaddig, amíg a szöveget lezáró nulla karakterhez nem érünk.

```
// Szöveg kirajzolása
char OLED_WriteString(char* str, FontDef Font, OLED_Color color)
{
    // Írás addig, amíg a pointer érvényes karakterre mutat
    while (*str)
    {
        if (OLED_WriteChar(*str, Font, color) != *str)
        {
            // A karaktert nem lehetett kiírni
            return (*str);
        }

        // Következő karakter
        str++;
    }

    // Minden rendben
    return (*str);
}
```

Példa

Elméletben már messzire jutottunk, itt az ideje, hogy a gyakorlatban is teszteljük tudásunkat! Ehhez segítségül összeraktunk egy kódrészletet a main.c fájlban, ahol tesztelni tudjuk az eddig megismert lehetőségek egy részét. A célunk az volt, hogy egy minimális ember-gép interfészt bemutató kis projekt jöjjön létre, ezért a kijelző mellé belecsempésztük a nyomógombok és a LED-ek használatát.

1.1.1.5 Az állapotgép

A fő programunk alapja egy állapotgép, melynek állapotaiban egy-egy példát láthatunk a kijelzőnk használatára. Az állapotok között a BTN2 (bal) és BTN4 (jobb) gombok segítségével tudunk váltani. Ezenkívül a középső gombhoz (BTN3) rendeltünk további funkciót, mégpedig a megjelenítőnk invertálásának ki-, illetve bekapcsolását. A gombok lenyomásának érzékelésekor visszajelzést adunk a LED-eken keresztül, hogy a programunk érzékelt egy megnyomást. Az állapotváltás viszont csak a nyomógombok elengedésekor fog megtörténni, mellyel egy időben az adott LED-et ki is kapcsoljuk.

Kezdeként pillantsunk rá az állapotgépünkre:

```
// Tesztfunkció kiválasztás
switch (test_state)
{
    case OLED_STATE_FONTS:           // Karakterkészlet megjelenítés
        OLED_TestFonts();
        break;
    case OLED_STATE_LINE:            // Véletlenszerű vonalgenerátor
        OLED_TestLine();
        break;
    case OLED_STATE_RECTANGLE:       // Téglalap alapú képernyővédő
        OLED_TestRectangle();
        break;
    case OLED_STATE_CIRCLE:          // Kör alapú képernyővédő
        OLED_TestCircle(&start_x, &start_r);
        break;
    case OLED_STATE_ARC:              // Körívek
        OLED_TestArc(&start_angle);
        break;
    case OLED_STATE_POLYLINE:        // Vonallánc
        OLED_TestPolyline();
        break;
    case OLED_STATE_CCE:              // Kristálytisza Elektronika
    default:
        OLED_TestCCE();
        break;
}
```

```
OLED_UpdateScreen();           // Kijelző frissítés
HAL_Delay(100);                 // Ciklusidő
```

Minden állapothoz egy-egy tesztfüggvény tartozik, mely egy rajzolat vagy kiíratást csinál az elnevezésnek megfelelően. A tényleges OLED kijelzőre küldést az állapotválasztás után végezzük el, hogy ezt csak egy helyen kelljen mindig meghívni. A 100 milliszekundum hosszú késleltetést azért raktam a legvégére, hogy a változások követhetőek legyenek. Láthatjuk, hogy az állapotoknak értelmezhető nevei vannak, amelyeket az alábbi helyen definiáltunk:

```
// Felsorolás az állapotgéphez
typedef enum
{
    OLED_STATE_CCE = 0x00,           // Kristálytisza Elektronika
    OLED_STATE_FONTS = 0x01,         // Karakterkészlet megjelenítés
    OLED_STATE_LINE = 0x02,          // Véletlenszerű vonalgenerátor
    OLED_STATE_RECTANGLE = 0x03,     // Téglalap alapú képernyővédő
    OLED_STATE_CIRCLE = 0x04,        // Kör alapú képernyővédő
    OLED_STATE_ARC = 0x05,           // Körívek
    OLED_STATE_POLYLINE = 0x06       // Vonallánc
} OLED_Test_state;
```

Nézzük a gombok és LED-ek kezelését:

```
while (1)
{
    // Gombkezelés - gombok állapotának beolvasása
    current_button_state[0] = HAL_GPIO_ReadPin(HMI_BTN_2_GPIO_Port,
    HMI_BTN_2_Pin);
    current_button_state[1] = HAL_GPIO_ReadPin(HMI_BTN_3_GPIO_Port,
    HMI_BTN_3_Pin);
    current_button_state[2] = HAL_GPIO_ReadPin(HMI_BTN_4_GPIO_Port,
    HMI_BTN_4_Pin);

    // Bal gomb (BTN2)
    if ((current_button_state[0] != previous_button_state[0]) && (GPIO_PIN_RESET ==
    current_button_state[0]))
    {
        if(OLED_STATE_CCE == test_state)
        {
```

```
        test_state = OLED_STATE_POLYLINE;
    }
    else
    {
        test_state -= 1;
    }

    // Képernyő újratöltés
    OLED_Fill(BLACK);
    OLED_UpdateScreen();
}

// Jobb gomb (BTN4)
if ((current_button_state[2] != previous_button_state[2]) && (GPIO_PIN_RESET ==
current_button_state[2]))
{
    if(OLED_STATE_POLYLINE == test_state)
    {
        test_state = OLED_STATE_CCE;
    }
    else
    {
        test_state += 1;
    }

    // Képernyő újratöltés
    OLED_Fill(BLACK);
    OLED_UpdateScreen();
}
```

Minden egyes ciklus elején beolvassuk a gombjaink állapotát, ami körülbelül 100 milliszekundumonként történik meg az előbbi késleltetésből kifolyólag. Miután megvannak a gombok aktuális állapotai, egyesével megvizsgáljuk azokat, hogy történt-e változás a korábbi állapothoz képest. Ha változott a gomb állapota és jelenleg nincs lenyomva, akkor a nyomógomb eddig le volt nyomva és most engedte fel a felhasználó. Ehhez az eseményhez szeretnénk állapotváltást hozzárendelni.

A bal oldali gomb az állapotváltozónk csökkentésére, a jobb oldali pedig a növelésre szolgál. Állapotváltáskor ügyelnünk kell a túlsordulás kezelésére (itt túlsordulás alatt az általunk definiált állapotokból való kifutást értem), valamint arra, hogy a kijelzőt le kell tisztázni, mielőtt bármi újat, bármi mást rajzolnánk rá.

A fel és le gomboknak (BTN1 és BTN5) jelen alkalmazásban nincsen funkciójuk.

A középső gomb az invertáltság módosítására szolgál:

- ha invertálva volt a kijelző színe, akkor bekapcsoljuk a normál megjelenítési módot
- normál működés esetén pedig bekapcsoljuk a kijelző színének invertálását

```
// Szín invertáló gomb (BTN3)

if ((current_button_state[1] != previous_button_state[1]) && (GPIO_PIN_RESET ==
current_button_state[1]))
{
    if(OFF == OLED_GetDisplayInverse())
    {
        // Inverz szín beállítás
        OLED_SetDisplayInverse(ON);
    }
    else
    {
        // Normál szín beállítás
        OLED_SetDisplayInverse(OFF);
    }
}
```

Ami még a kívánt funkcionalitás megvalósításához szükséges, az a gombok megnyomásának visszajelzése a LED-eken keresztül, valamint a gombok állapotának elmentése.

```
// Gombok állapotának monitorozása a LED-eken

HAL_GPIO_WritePin(HMI_LED_2_GPIO_Port, HMI_LED_2_Pin, current_button_state[0]);
HAL_GPIO_WritePin(HMI_LED_3_GPIO_Port, HMI_LED_3_Pin, current_button_state[1]);
HAL_GPIO_WritePin(HMI_LED_4_GPIO_Port, HMI_LED_4_Pin, current_button_state[2]);

// Aktuális gomb állapotok elmentése

previous_button_state[0] = current_button_state[0];
previous_button_state[1] = current_button_state[1];
previous_button_state[2] = current_button_state[2];
```

Utoljára maradt a kijelzőt működtető tesztfüggvények átnézése mind kód szinten, mind pedig vizuálisan. Haladjunk az állapotok sorrendjében:

1.1.1.6 1. OLED_STATE_CCE

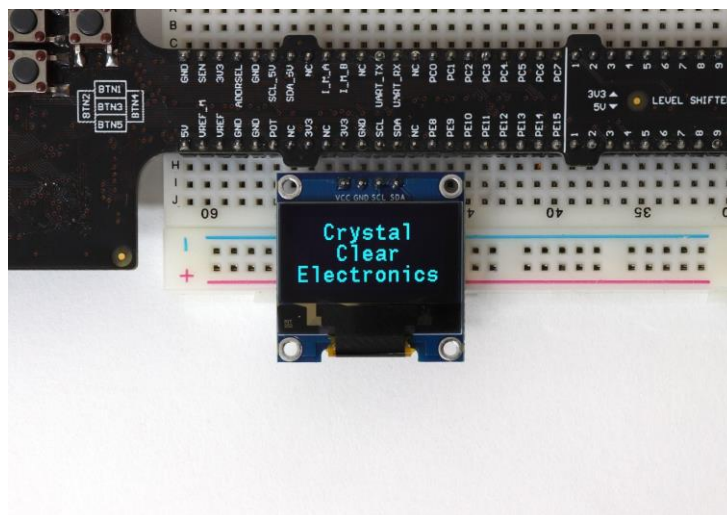
Ez a függvényünk a Kristálytisza Elektronika angol nevét írja ki a kijelzőre: Crystal Clear Electronics.

```
// Kristálytisza Elektronika kiírása
void OLED_TestCCE()
{
    OLED_SetCursor(25, 5);
    OLED_WriteString("Crystal", Font_11x18, WHITE);
    OLED_SetCursor(36, 5+18);
    OLED_WriteString("Clear", Font_11x18, WHITE);
    OLED_SetCursor(3, 5+18+18);
    OLED_WriteString("Electronics", Font_11x18, WHITE);

    return;
}
```

A SetCursor() segítségével be lehet állítani a szöveg pozícióját, amit ebben az esetben középre igazítottunk. A szöveg hosszából (karakterek száma * karakter szélesség) és a kijelző szélességéből számítható a pozíció.

Vizuális eredmény:



36. ábra: OLED_TestCCE() által megjelenített adatok

Az állapotgépünk alapállapotának (angolul: default state) ezt az állapotot adtuk meg, ami azt jelenti, hogy ha valamilyen okból a *test_state* változónk egy előre nem definiált értéket venne fel, akkor ez az ág fog végrehajtódni.

Jobb gombot nyomva a fejlesztőkártyán át is válthatunk a következő állapotra.

1.1.1.7 2. OLED_STATE_FONTS

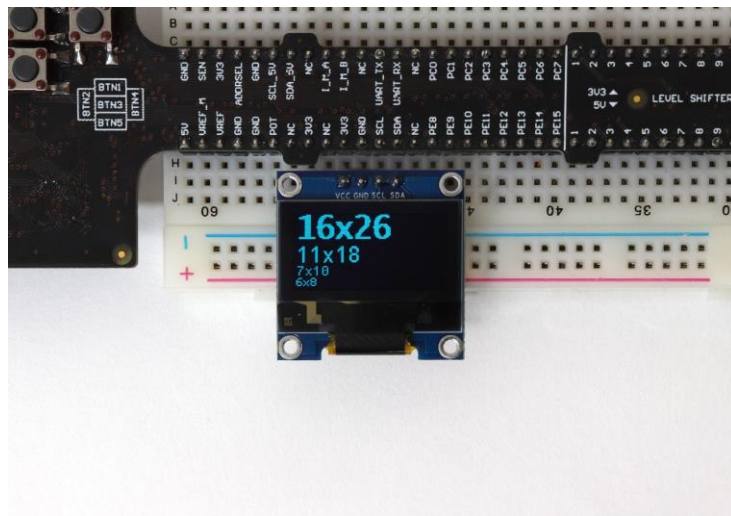
Ez a teszt a betűtípusok lehetséges méreteit hivatott bemutatni. Négy karakterkészletünk van, amelyeket megjelenítünk a képernyőn, így azok méretbeli különbségei is könnyedén láthatók.

```
// Karakterkészlet megjelenítés
void OLED_TestFonts()
{
    OLED_SetCursor(2, 0);
    OLED_WriteString("16x26", Font_16x26, WHITE);
    OLED_SetCursor(2, 26);
    OLED_WriteString("11x18", Font_11x18, WHITE);
    OLED_SetCursor(2, 26+18);
    OLED_WriteString("7x10", Font_7x10, WHITE);
    OLED_SetCursor(2, 26+18+10);
    OLED_WriteString("6x8", Font_6x8, WHITE);

    return;
}
```

Szöveggént a karakterek méreteit jelenítjük meg balra igazított elrendezésben.

Vizuális eredmény:



37. ábra: OLED_TestFonts() által megjelenített adatok

1.1.1.8 3. OLED_STATE_LINE

A vonalak megjelenítését igyekeztünk kicsit érdekesebbé tenni azzal, hogy egy dinamikusan változó képernyőt kapunk eredményül.

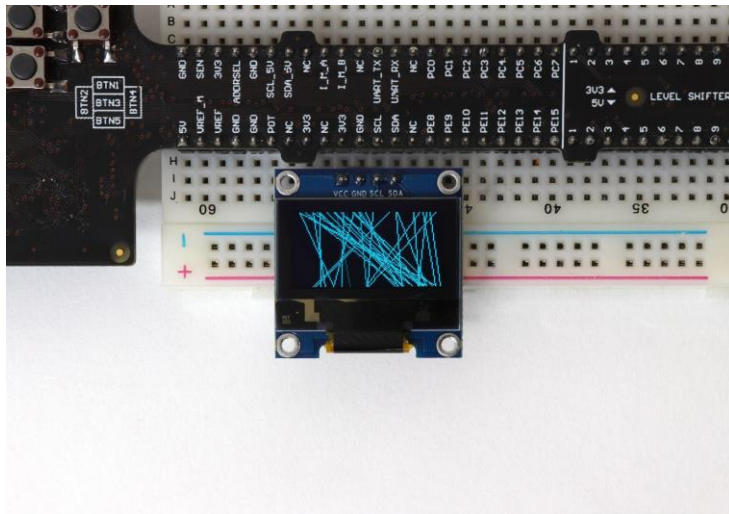
```
// Véletlenszerű vonalgenerátor
void OLED_TestLine()
{
    int random_x1 = rand() % 128; // random szám 0 - 127 között
    int random_x2 = rand() % 128; // random szám 0 - 127 között

    OLED_DrawLine((uint8_t)random_x1, 0, (uint8_t)random_x2, 63, WHITE);

    return;
}
```

Maga a kód pár sorból áll csupán, mégis az eredmény rejt némi játékoságot. Az ötlet az, hogy a képernyő felső és alsó szélei között véletlenszerű vonalakat húzunk. Ehhez generálunk két véletlen számot 0 és 127 között, melyeket a kirajzolandó vonalunk x koordinátáiként használunk fel. Az y koordinátákra konstansként tekintünk, tehát rögzítjük a felső (0. sor) és alsó (63. sor) sorokat. Nincs más hátra, mint kirajzolni a vonalunkat. Minél többször hívjuk meg ezt a függvényt, annál több vonal rajzolódik ki a képernyőre.

Vizuális eredmény:



38. ábra: `OLED_TestLine()` által megjelenített adatok

1.1.1.9 4. `OLED_STATE_RECTANGLE`

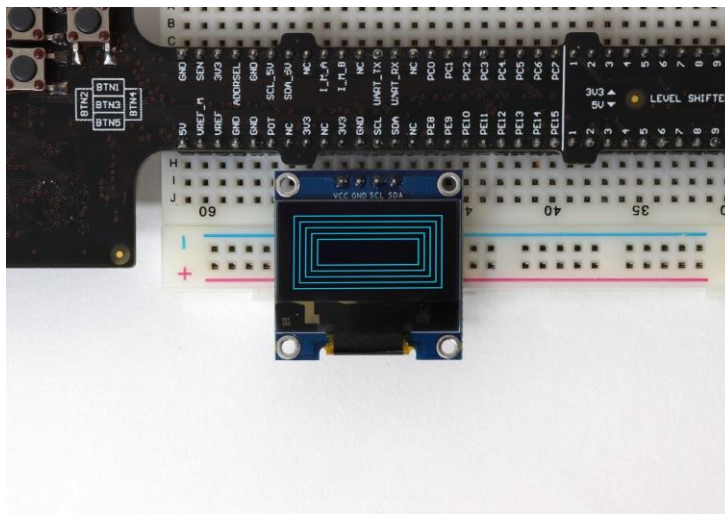
Ebben a példában öt téglalapot rajzolunk ki egy ciklus felhasználásával, amelyeknek szomszédos oldalai egyenlő távolságra vannak egymástól.

```
// Téglalap alapú képernyővédő
void OLED_TestRectangle()
{
    uint32_t delta;

    for (delta = 0; delta < 5; delta++)
    {
        OLED_DrawRectangle(1 + (5*delta), 1 + (5*delta) , OLED_WIDTH-1 -
        (5*delta), OLED_HEIGHT-1 - (5*delta), WHITE);
    }

    return;
}
```

Vizuális eredmény:



39. ábra: `OLED_TestRectangle()` által megjelenített adatok

1.1.1.10 5. `OLED_STATE_CIRCLE`

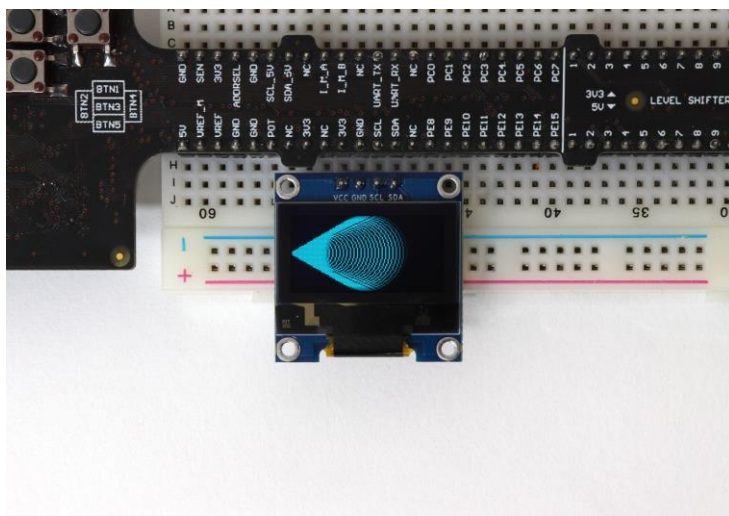
Ebben az esetben is a lehetőségekre szerettünk volna rávilágítani. Példánkban köröket rajzolunk a kijelző vízszintes középvonalán haladva balról jobbra úgy, hogy a körök átmérőjét lépésről lépésre növeljük.

```
// Kör alapú képernyővédő
void OLED_TestCircle(uint8_t* x, uint8_t* r)
{
    OLED_DrawCircle(*x, 31, *r, WHITE);
    *r += 1;
    *x += 2;

    if (100 < *x)
    {
        *x = 0;
        *r = 1;
        OLED_Fill(BLACK);
    }

    return;
}
```

Vizuális eredmény:



40. ábra: OLED_TestCircle() által megjelenített adatok

1.1.1.11 6. OLED_STATE_ARC

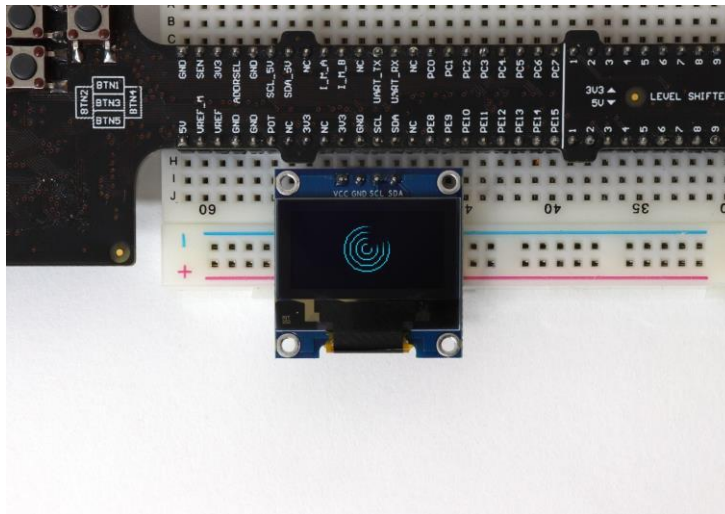
Megismerkedhetünk négy darab szinkronban forgó körívvel, melyek minden lépésben 10 fokkal fordulnak el.

```
//Körívek
void OLED_TestArc(uint16_t* angle)
{
    OLED_Fill(BLACK);
    OLED_DrawArc(63, 31, 5, *angle, 270+(*angle), WHITE);
    OLED_DrawArc(63, 31, 10, *angle, 270+(*angle), WHITE);
    OLED_DrawArc(63, 31, 15, *angle, 270+(*angle), WHITE);
    OLED_DrawArc(63, 31, 20, *angle, 270+(*angle), WHITE);

    *angle += 10;

    return;
}
```

Vizuális eredmény:



41. ábra: OLED_TestArc() által megjelenített adatok

1.1.1.12 7. OLED_STATE_POLYLINE

Végül, de nem utolsósorban, egy példát láthatunk a sokszögek rajzolásának módjára.

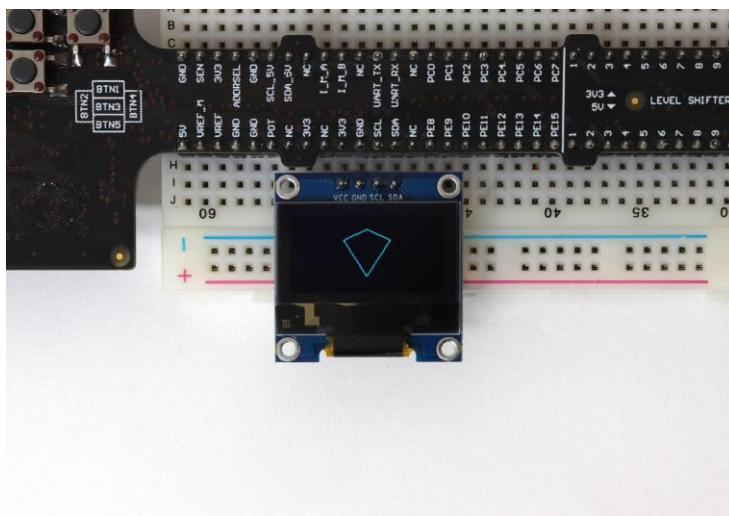
```
// Vonallánc
void OLED_TestPolyline()
{
    OLED_Vertex loc_vertex[] =
    {
        {63,11},
        {43,20},
        {63,51},
        {83,20},
        {63,11}
    };

    OLED_DrawPolyline(loc_vertex, sizeof(loc_vertex)/sizeof(loc_vertex[0]), WHITE);

    return;
}
```

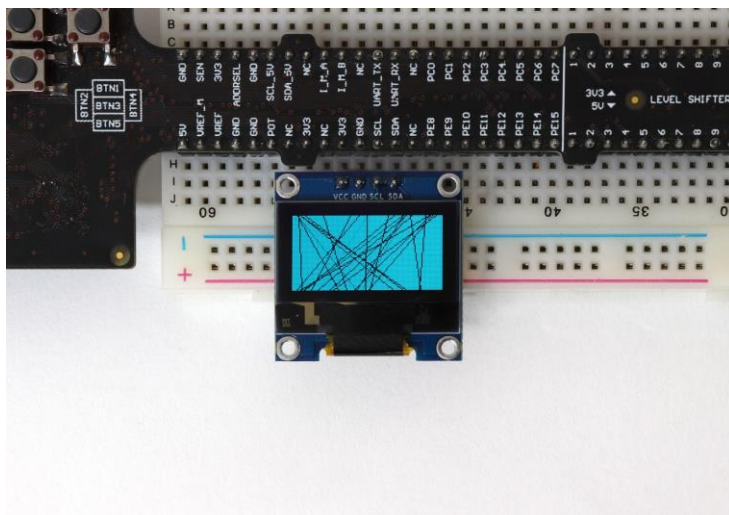
Egy deltoidot ábrázoltunk, amelyhez megadtuk csúcsainak koordinátáit. A kiindulópontot zárópontként is meg kell adni, hogy a deltoidunk egy befejezett, zárt alakzat legyen.

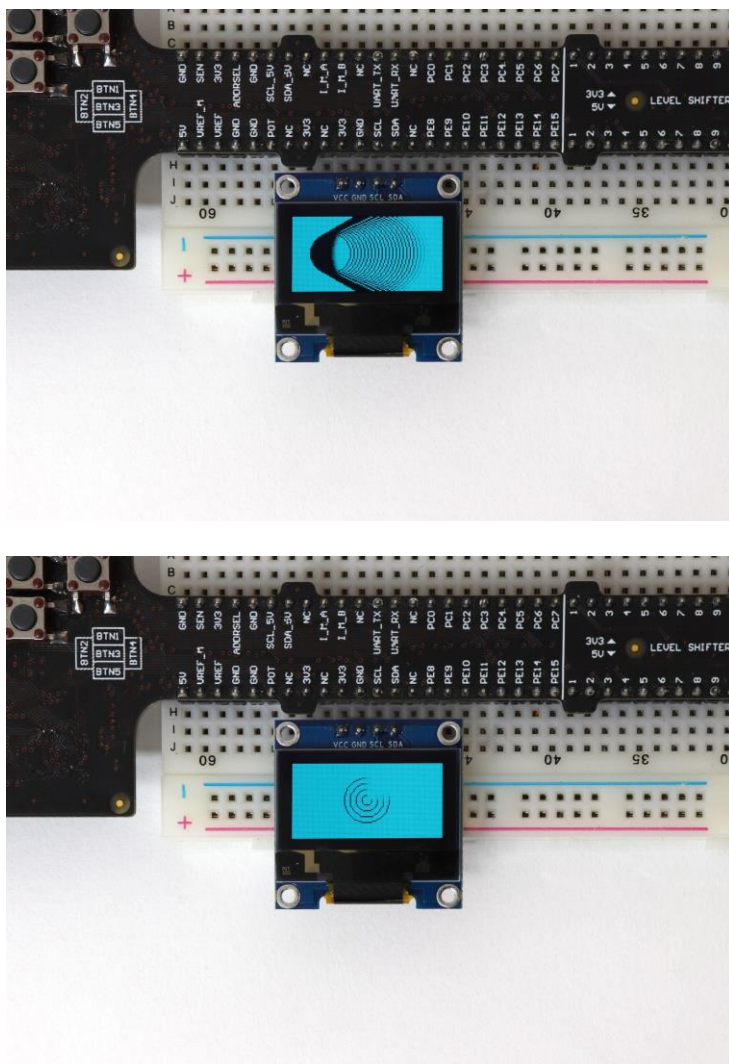
Vizuális eredmény:



42. ábra: OLED_TestPolyline() által megjelenített adatok

Ha eddig nem tettétek meg, akkor invertáljátok meg a képernyő színeit, és lépkedjünk újra végig a példákön, hogy láthassuk a különbséget. Az animációk közben is nyugodtan használjátok ezt a lehetőséget. Néhány kép a hatásáról:





43. ábra: Képernyő színeinek invertálása

A fenti minták példaként szolgálnak, bátran próbáljátok ki, hogy a paraméterek változtatása milyen hatással van a vizuális eredményre! Még jobb, ha saját feladatot találtok ki magatoknak.

Nem jut eszetekbe új példa? Néhány megvalósításra váró ötletet összegyűjtöttünk a számotokra:

- Véletlenszerű helyekre, véletlenszerű méretű körök rajzolása
- Véletlenszerű vonalak a teljes képernyőn
- Rajzolj minél több szabályos sokszöget a képernyőre: háromszög, négyzet, ötszög, hatszög stb.
- Egyszerűbb játékok is megvalósíthatóak, például elég csak a Snake-re gondolni.

ÖSSZEFOGLALÁS

Ebben a tananyagrészben megtudtuk, hogy mit értünk ember-gép interfészek alatt, majd csoportosítottuk azokat be- és kimenetek szerint. Sorra vettük a fejlesztőkártyán lévő, és az ahhoz járó bemeneteket, melyek segítségével információt adhatunk a gépek számára. Ezt követően a kimeneti interfészekre helyeztük a hangsúlyt, mégpedig azokra a típusokra, amelyek vizuális úton képesek az embernek adatot szolgáltatni. A LED-ektől gyorsan eljutottunk a kijelzők világáig, amelyek zárásaként

sorra vettük az alapvető megjelenítőket, melyekkel nagy eséllyel találkozhattok a saját projektjeitek során. Kitértünk az eszközök mögött rejtőzködő technológiai megvalósításokra, a hardveres interfészekre, illetve a szoftveres vezérelhetőségekre. A csomaghoz járó grafikus OLED kijelzőt pedig sikerült együtt működésre bírni és részletesen programozásában részletesen elmerülni.

Remélem élveztétek a közös utunkat, további sok sikert kívánok a folytatáshoz!